aerospace

MDPI

Article

Application of DRL-Based Algorithm for the Resolution of Strategic Conflicts in U-Space Airspaces †

Manuel González *, Sandra Amarillo ‡, Alex Sanchis ‡ and Juan Vicente Balbastre ‡

Air Navigation Systems Research Group, ITACA Institute, Universitat Politècnica de València, 46022 València, Spain; sanamva@upv.edu.es (S.A.); asanmar4@upv.edu.es (A.S.); jbalbast@upv.edu.es (J.V.B.)

* Correspondence: mgonrod1@upv.edu.es

† Presented at the 15th EASN International Conference, in Madrid, Spain, from 14–17 of October 2025.

‡ These authors contributed equally to this work.

Abstract

The rapid expansion of Unmanned Aircraft Systems (UAS) operations has created an urgent need for scalable strategic conflict resolution methods within the U-space framework. When requested 4D flight plans overlap with previously authorised ones, the Flight Authorisation Service denies the request and can provide the UAS operator with an alternative, conflict-free route. While traditional pathfinding algorithms ensure optimal routes, their computational cost creates a critical bottleneck during the flight activation phase or emergency missions, which demand near-instantaneous responses. To address this, we propose a three-stage framework. First, an Octree spatial partitioning discretises the airspace to identify occupied cells. Second, both A* and JPS algorithms are implemented to establish an optimal reference route. Finally, a standard Deep Reinforcement Learning (DRL) model, trained on realistic PX4 Simulator trajectories and using a well-adjusted reward function, generates alternative paths that optimise distance and energy. Results demonstrate that this DRL architecture achieves near-optimal routing behaviour. Crucially, it reduces computation time by several orders of magnitude compared to traditional algorithms, solving complex conflicts in milliseconds rather than seconds. We conclude that simple, well-tuned DRL architectures overcome latency limitations of classical pathfinding while achieving optimal results, ensuring rapid, safe, and efficient conflict resolution for high-density U-space.

Keywords: PX4; U-space; Machine Learning; strategic resolution; Flight Authorisation Service

Check for updates

Academic Editors: Spiros Pantelakis, Andreas Strohmayer and Gustavo Alonso-Rodrigo

Received: 27 February 2026
Revised: 30 May 2026
Accepted: 1 June 2026
Published: 3 June 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Although Unmanned Aircraft Systems (UAS) have been used in military operations since the early 20th century—including World War I [1] and the Vietnam War [2]—widespread interest in their civil applications did not emerge until the beginning of the 21st century. Since then, the technology has experienced rapid growth, driven by expectations of numerous professional applications across diverse sectors such as urban mobility, parcel delivery, public safety and security, entertainment, agriculture, and infrastructure inspection, in addition to recreational use. Most of these operations are envisaged in the Very-Low-Level airspace (below 500 ft) over urban areas beyond the visual line-of-sight (BVLOS) from the remote pilot. This type of operation poses a challenge to civil aviation authorities, since they are not covered by visual flight rules (VFRs) and instrumental flight rules (IFRs) used in manned aviation [3].

In this context, the concept of UAS Traffic Management (UTM) emerged in the mid-2010s as a dedicated component of air traffic management aimed at enabling the safe, efficient, and economically sustainable integration of UAS operations. According to the International Civil Aviation Organization (ICAO) [4], UTM comprises the set of services and operational functions required to ensure the seamless integration of UAS into the airspace, encompassing both airborne and ground-based elements.

The European implementation of UTM, established through Commission Implementing Regulation (EU) 2021/664 on a regulatory framework for the U-space [5], introduces the concept of U-space airspace, defined as a UAS geographical zone within which UAS operations are permitted only through the use of a predefined set of U-space services. These services include, at a minimum, network identification, flight authorisation, geo-awareness, and traffic information services. In addition, specific geographical zones in which UAS operations may be prohibited or restricted can be designated within a U-space airspace.

The Flight Authorisation Service (FAS) constitutes a core safety mechanism for UAS operations within U-space airspace. UAS operators submit Flight Authorisation Requests (FARs) to the FAS, each including the intended UAS trajectory represented as a sequence of four-dimensional (4D) volumes that account for flight execution uncertainties up to a 95% probability bound. Upon receipt, the FAS applies predefined spatial and temporal deviation thresholds, introducing a safety buffer that supports an appropriate balance between risk mitigation and efficient airspace utilisation. The FAS subsequently assesses whether the 4D trajectory associated with the newly submitted FAR generates any spatial or temporal intersections with previously authorised 4D trajectories and verifies compliance with applicable geographical restrictions, including prohibited UAS geographical zones. If no conflicts are identified and airspace capacity constraints are respected, the request is authorised. In the event that an overlap is detected, the FAS may propose an alternative conflict-free 4D trajectory to the UAS operator [5].

FARs with the same priority level are processed on a first-come, first-served basis. However, high-priority FARs may override previously authorised lower-priority requests. Consequently, the introduction of high-priority operations can significantly affect already approved trajectories. To mitigate this impact, the FAS may propose a 4D trajectory for the high-priority operation that minimises disruption to previously authorised flights while still satisfying the mission-specific priority requirements. While standard UAS operations are typically planned well in advance of take-off, priority missions—such as the urgent delivery of medical supplies—require near-immediate authorisation.

In all cases, once a FAR has been approved by the FAS, the UAS operator is required to request activation of the authorisation shortly before take-off. According to the European regulatory framework [6], the time elapsed between the activation request and the confirmation of flight activation shall not exceed 5 s in 95% of cases. Prior to issuing the activation, the FAS performs a final pre-departure verification to ensure that the authorised 4D trajectory remains compatible with other ongoing operations and with applicable airspace constraints.

One of the fundamental safety mechanisms in U-space airspace is the segregation of manned aircraft operating under Air Traffic Control (ATC) instructions through Dynamic Airspace Reconfiguration (DAR). A DAR entails the temporary deactivation of part of, or the entirety of, a U-space airspace. It is triggered when the relevant Air Traffic Services (ATS) unit determines that a manned aircraft under ATC control is about to enter the U-space airspace. Upon activation of a DAR, the FAS shall reject any FAR containing 4D trajectories that intersect the affected airspace volume, withdraw previously issued flight authorisations impacted by the DAR, and update the trajectories of affected airborne UAS to ensure that they vacate the impacted volume and either safely complete their mission or perform a controlled landing.

The strategic deconfliction process carried out by the FAS is not posed as a continuous-time optimal control problem, but as an intersection-free allocation of 4D airspace volumes associated with flight authorisations and UAS geographical zones, as stated in [5]. Therefore, the key computational aim is efficient 4D occupancy management and collision queries rather than continuous trajectory integration.

Standard Specification for UAS Traffic Management (UTM) UAS Service Supplier (USS) Interoperability, ASTM F3548-21 also reinforces the idea of the discretisation of the airspace. It proposes that the implementation of the airspace representation can be conceptualised as a grid, and mapping a flight into the airspace representation determines which grid cells it intersects [7]. In line with this approach, we propose an octree-based discretisation of the U-space into cells, following Amarillo, S. et al. [8], to provide an efficient airspace representation aligned with the ASTM F3548-21 standard.

In this context, the projected growth in demand for Unmanned Aircraft Systems (UAS) across multiple operational domains has generated an urgent need to effectively manage the emerging U-space airspace in order to ensure safety, scalability, and operational efficiency. To address this challenge, research efforts are being conducted worldwide, particularly at the intersection of two rapidly evolving technological fields: UAS and Artificial Intelligence (AI) [9].

This work follows a three-stage pipeline: First, the tool generates an Octree-based 4D graphical representation of the occupied airspace, enabling operators to visually identify candidate routes. Second, both an A* (A-star) and a JPS pathfinding algorithm (only in 2D) are applied in the discretised space to obtain a conflict-free reference trajectory. Third, a Reinforcement Learning (RL) approach is introduced as an alternative to compute near-optimal routes at lower computational cost, and compared with the A* and JPS reference trajectories, as an indicator of its performance.

To ensure that the learning-based planner is trained and evaluated under operationally meaningful conditions, the proposed framework relies on realistic trajectory data rather than purely geometric or kinematic toy models. Training a DRL agent on straight-line segments with artificial noise may lead to policies that perform well in simplified settings but fail to capture the behaviour of real UAS operations, where the executed path is shaped by autopilot logic, vehicle dynamics, and environmental disturbances.

Therefore, we generate realistic UAS trajectories using a PX4 Software-In-The-Loop (SITL) pipeline coupled with Gazebo, which emulates autopilot-driven flight behaviour and enables the systematic injection of operational disturbances and off-nominal events (e.g., wind effects, link disruptions and failsafe behaviours) [10]. This approach provides higher-fidelity datasets for U-space systems assessment and increases the credibility and potential generalisation of the learned policy when compared to trajectories built from idealised motion assumptions. In addition, the use of fast-time simulation enables large-scale dataset generation across heterogeneous scenarios, alleviating the current lack of widely adopted performance models for UAS trajectories (i.e., an analogue of BADA for UAS).

As for 2D or 3D resolution, we adopt a hierarchical planning strategy: a primary 2D planner operates at cruise altitude, consistent with common UAS operational profiles, since they are usually planned to reach cruise altitude and maintain it until descent, thereby maintaining energetically efficient trajectories. On the other hand, a secondary 3D planner is triggered only when no feasible 2D solution exists due to airspace congestion. Provided that 3D resolution can infer energetically costly movements, the proposed 3D model is evaluated not only in terms of distance but also in terms of energy. The 2D resolution is usually more efficient in terms of energy and computational efficiency. This two-layer design prioritises fast, energy-efficient planning while preserving completeness when 2D becomes infeasible.

The adoption of A* and JPS as the benchmarks for this paper is based on A* being the gold standard for evaluating advanced pathfinding methods in UAS path planning. It is widely considered as the reference method in Static Optimal Search [11]. And, even though it is a classical method and was created by Hart, P. et al., in 1968, in A Formal Basis for the Heuristic Determination of Minimum Cost Paths [12], research is still being done in order to address its weaknesses in UAS path planning [13]. JPS is just one of the advances of the A* approach, explained later in Section 3.7.

However, the use of DRL is undoubtedly questionable. Although it is possible to achieve a near-optimal result (compared to A* and JPS) while reducing computation time, on some occasions, it may not be extremely decisive. In time, two different stages coexist when it comes to flight planning:

First, the approval stage. In this part, flight plans are requested for approval. Although EASA mentions in Easy Access Rules for U-space [6] that the time required to approve a flight authorisation request should be as short as practically possible, this stage typically takes place from hours to days before the flight. Provided that a typical resolution might take up to seconds in A* and JPS, and centiseconds in DRL, there may be no tangible differences. Of course, every second matters, but not as importantly as in the next stage.

Secondly, the activation stage comes into play. The flight's activation occurs at the exact moment it starts. In the time between the previous and the present stage, the state of the U-space airspace is likely to change, e.g., new authorisations, new restricted geographical zones, and changing wind conditions. In this stage, a nearly immediate response is required (Standard F3548-21 states that the activation of a flight request should be made within n seconds in 95% of the time [7] and EASA, in Easy Access Rules for U-space interprets that it should be made, e.g., within 5 s [6]); therefore, DRL outperforms A* and JPS in this aspect and is essential to maintain U-space airspace efficiency [6].

Furthermore, if a high-priority request arrives, e.g., an emergency mission, those flights that interfere with the emergency flight shall be cancelled. However, if the DRL tool can provide a free-of-conflict route instantly, there is no need to cancel the previously accepted flights, and airspace efficiency would not be impacted.

Finally, the proposed framework is evaluated in terms of feasibility, optimality with respect to the A* and JPS references, computational latency and (in the 3D fallback) energy-related cost. Results show that the Reinforcement Learning-based planner can deliver absolutely optimal deconflicted routes (in terms of cells) while substantially reducing response time, which is particularly relevant for the activation stage under dynamically changing U-space conditions. Unlike many existing studies that rely solely on A* for benchmarking, the inclusion of JPS as a reference provides a more rigorous validation of the proposed framework's efficiency.

The structure of this paper follows a systematic approach, beginning with a Literature Review section (Section 2) that presents recent, high-impact work in this field. Next, the Materials and Methods section (Section 3) describes in detail the resources and procedures used throughout the study, including Airspace Discretisation, UAS Geographical Zones, the energy model, and the operation of the A* and JPS (used to generate reference routes) and RL. Afterwards, the Deep Reinforcement Learning Models section (Section 4) outlines the design choices and training workflow of the proposed RL models. Then, the Results section (Section 5) presents and analyses the performance of the proposed models across a comprehensive set of use cases that cover the full range of operational requirements for flight authorisation services, as defined by the EU regulatory framework introduced earlier, and compares the performance of the RL approach with those of A* and JPS, followed by the Discussion section (Section 6), where the feasibility of the tool is analysed. Then, the Conclusions (Section 7) highlight the main contributions and future directions of this work.

To close, the Future Work section (Section 8) mentions some improvements or alternatives that might be worth researching in the future.

2. Literature Review

The rapid proliferation of Unmanned Aerial Systems (UAS) and the imminent integration of these vehicles into urban environments (Urban Air Mobility, UAM) have driven a transformation in airspace management. Forecasts indicate that the traffic volume of UAS in low-altitude airspace will be much higher than the historical density of commercial aviation [4], necessitating a change in the paradigm of airspace control, which has historically been centred on humans. To ensure the U-space's scalability is not compromised, it appears necessary to digitalise U-space services. More specifically, this paper pivots around the strategic conflict resolution service (SCRS), which offers an alternative optimal route when a flight plan has been withdrawn due to an overlap with a previously accepted flight or a restricted spatial volume. The SCRS is described in the European UTM ConOps (known as U-space) [14] and is one of the optional features of the regulated flight authrorisation service in [5].

Traditionally, in path planning, classic algorithms have been the most widely used. Graph-based algorithms such as A* or Dijkstra approach the airspace as a discrete grid and evaluate the transitions between nodes, accumulating costs and applying heuristic functions [15]. A* algorithm, despite being widely considered as the golden standard to guarantee global optimality in terms of distance, suffers from an asymptotical computational complexity, which in the worst case, grows as $\mathcal{O}(b^d)$, where b represents the branching factor and d the length of the route (see Equation (4)). This exponential growth suggests that for extensive flight routes that cross long, significant urban areas, the computational time required exceeds the time limit for the activation-phase path planning. However, traditional approaches have also enhanced speed, such as Jump Point Search [16], presented in Section 3.7.

While prior work such as Wu, Q. et al., in 2020, in A new fallback beetle antennae search algorithm for path planning of mobile robots with collision-free capability [17] and Chen, D. et al., in 2023, in A Novel Convolutional Neural Network Model Based on Beetle Antennae Search Optimization Algorithm for Computerized Tomography Diagnosis [18] demonstrate the flexibility of beetle antennae search (BAS)-based methods in optimisation problems, these stochastic metaheuristics are primarily designed for continuous search spaces and do not provide deterministic optimality guarantees. In contrast, the high-density discrete U-space grid considered in this work requires strict guarantees on path optimality, reproducibility, and constraint satisfaction to comply with operational and regulatory requirements. Furthermore, BAS/FAS methods typically involve iterative, parameter-sensitive optimisation processes, making fair and reproducible benchmarking challenging without extensive tuning, which falls beyond the scope of this study. For these reasons, we prioritise A* and JPS as deterministic shortest-path baselines and DRL for real-time adaptive conflict resolution. Given that the primary objective of this paper is to evaluate DRL performance under operational constraints, these methods provide a more appropriate and representative benchmark set.

However, in these extensive data environments, not only must the solving method be fast enough, but the management of the discretised airspace should also be efficient enough to prevent discretisation from becoming the resolution bottleneck. Thus, the scalability of this path-planning resolution strongly relies on efficient spatial indexing and filtering. Conventional geometric comparison methods, which scale at $\mathcal{O}(n^2)$, are being replaced by three-dimensional Morton (Z-curve) and Octree spatial discretisation. In 2026, Li, Q. et al., in Research on Multi-Airspace Conflict Detection and Resolution Method Based on Octree

Spatial Grid Partitioning, proposed an efficient approach to provide reliable solutions based on Octree [19]. Furthermore, in 2025, Amarillo, S. et al., in Proposal of UAS Strategic Conflict Detection Concept with a Centralised Service in Multi-USSP Environment Using an Octree Data Structure [8], presented an algorithm that aligns with the standard ASTM F3548-21 [7], and successfully demonstrated the impact of the developed solution. This approach has been adopted to discretise the U-space airspace in this work.

As for the integration of Artificial Intelligence in U-space, more specifically, Machine Learning models, advances have been made. In 2024, Gordo, V. et al., in Feasibility of Conflict Prediction of Drone Trajectories by Means of Machine Learning Techniques [20] demonstrated that integrating high-density U-space structures with ML classifiers, such as Neural Networks (NN), allows for the prediction of collisions up to 100 s in advance with high precision, showing the advantages of using ML in terms of computational time in U-space, paving the way for the integration of this technology in Strategic Conflict Detection and Resolution.

More specifically, in Conflict Resolution, although the A* algorithm remains a benchmark for global optimality, its computational complexity grows exponentially—see Equation (4)—leading to significant latency issues. Further advances have been made in order to make the most of it, e.g., in 2024, He, Y. et al., in A new method for unmanned aerial vehicle path planning in complex environments [21] developed an improved A* for UAS Route Planning, enhancing its computational efficiency.

The current research not only focuses on improving classical algorithms, but also on finding suitable and robust alternatives. Even quantum technologies are starting to be used. Innan, N. et al. developed a Quantum-Assisted Path Planner for UAS Navigation, comparing its performance with A*, showing promising results [22].

However, the main alternative technologies being researched and the most promising are AI-based. The research community is transitioning from classical search-based algorithms to Reinforcement Learning (RL) to address the "computational complexity explosion" in pathfinding across complex, extensive 3D volumes. RL is one of the favourites, and Q-Learning is one of the most widely used RL algorithms for this purpose. Nevertheless, it has difficulties generalising in large state spaces. In 2013, Mnih, V. et al., in Playing Atari with Deep Reinforcement Learning [23] proposed combining Q-Learning with Deep Neural Networks (Deep Q Network - DQN) to address the aforementioned problem in discrete environments, creating a reference technique that is widely used in this day and age. In a 2D discrete environment, in 2017, Wang. Z et al., in Deep Reinforcement Learning Assisted UAV Path Planning Relying on Cumulative Reward Mode and Region Segmentation [24], proposed a Deep Reinforcement Learning (DRL) Model for UAS Path Planning, based on DQN, showing its robustness in solving path planning problems in discrete environments, along with its computational advantages in comparison with A*. In a more recent work, in 2023, Westheider, J. et al., in Multi-UAV Adaptive Path Planning Using Deep Reinforcement Learning [25], empirically demonstrated that DRL can outperform state-of-the-art non-learning-based methods. Furthermore, in 2025, Sharma, G. et al., in Deep Reinforcement Learning-Based Framework for Path Planning of AUAVs [26], presented a Soft Actor-Critic (SAC) method and demonstrated its feasibility for 2D path planning.

Complex DRL architectures, such as D3QN and PPO, exist and have proven successful in UAS path planning; e.g., in 2025, Jarray, R. et al., in Improved deep Q-network-based UAV path planning in partially observable environments [27], proposed an innovative variant based on Improved DQN for partially observable scenarios. Despite all these advances, the recent literature suggests that the performance bottleneck in RL path planning is often not the architecture of the whole model itself, but rather the optimisation of hyperparameters (e.g., Learning Rate) and the design of the reward signal. In 2024,

Dierkes, J. et al., in Combining Automated Optimisation of Hyperparameters and Reward Shape [28] empirically demonstrated that the learning rate (which is one of the most complex hyperparameters) and the reward shape are mutually dependent, and that a standard RL agent with finely tuned hyperparameters can sharply outperform more complex models that lack this optimisation. Based on the literature survey presented in this section, this work adopts a standard DQN architecture for path planning and evaluates its performance against established deterministic baselines such as A* and JPS.

3. Materials and Methods

3.1. UAS Trajectories Generation and Fidelity

The accurate assessment of U-space services requires high-fidelity trajectory data. The aviation industry currently lacks a standardised performance model for Unmanned Aircraft Systems (UAS) comparable to the Base of Aircraft Data (BADA) [29] used in manned aviation. Furthermore, large-scale empirical datasets of autonomous drone flights in urban environments are virtually nonexistent. Consequently, trajectory modelling often relies on simplified kinematic approaches, such as Uniform Rectilinear Motion (MRU) models appended with random directional noise. These basic models fail to capture the complex physical realities of UAS flight, omitting critical factors like multirotor inertia, aerodynamic drag, environmental disturbances, and continuous autopilot corrections. Recent studies emphasise that multirotors operating in low-altitude urban environments are highly susceptible to wind gusts, which cause significant deviations from the intended flight path that simple kinematic models cannot predict [30].

To overcome these limitations, in this study we employ a high-fidelity Software-in-the-Loop (SITL) simulation environment utilising the PX4 open-source flight control software paired with the Gazebo physics engine [10]. This approach effectively bridges the simulation-to-reality gap by executing the exact flight control stack, guidance laws, and navigation algorithms that run on physical hardware. Thus, the resulting trajectories represent physically constrained flight executions rather than idealised geometric paths.

To illustrate the necessity of this high-fidelity approach, Figure 1 demonstrates the temporal evolution of spatial deviation between two identical flight plans executed under different environmental conditions (nominal versus constant wind). Unlike simplified MRU models that inject basic statistical noise, the error introduced in the PX4 SITL/Gazebo environment is the direct result of complex physical interactions.

As observed in Figure 1, the deviation exhibits an increasing trend during the cruise phase, punctuated by sharp, periodic corrections as the autopilot actively compensates upon reaching specific waypoints. The most significant irregularities manifest during transitional flight phases. The takeoff phase shows a steep initial deviation as wind resistance counters the vertical ascent, while the landing phase produces the peak deviation due to lateral drift induced during a gradual descent. Comparing these physically constrained trajectories to a basic MRU model would reveal massive positional discrepancies, rendering simplified kinematic models inadequate for rigorous U-space regulatory validation.

For this study, we generated a baseline dataset of ten straight cruise trajectories subjected to nominal default wind conditions. This setup forces the PX4 autopilot to actively compensate for aerodynamic drift, creating realistic micro-deviations and speed fluctuations. We subsequently use these baseline executions as the structural foundation to build the pseudo-random trajectories evaluated later in this work. Generating these trajectories requires a robust automation infrastructure. We developed a Python (v3.12) -based toolchain leveraging the MAVSDK library to autonomously upload flight plans and execute missions. To resolve native PX4 UDP port allocation bottlenecks and ensure scalability, we encapsulated the entire simulation environment within Docker containers. This containerised architecture isolates

each process, preventing cross-talk and bypassing port conflicts. Consequently, the system dynamically allocates resources to run multiple trajectory simulations concurrently, drastically reducing computation time while continuously exporting high-resolution telemetry data (3D position, velocity, and precise orientation) to a central database.

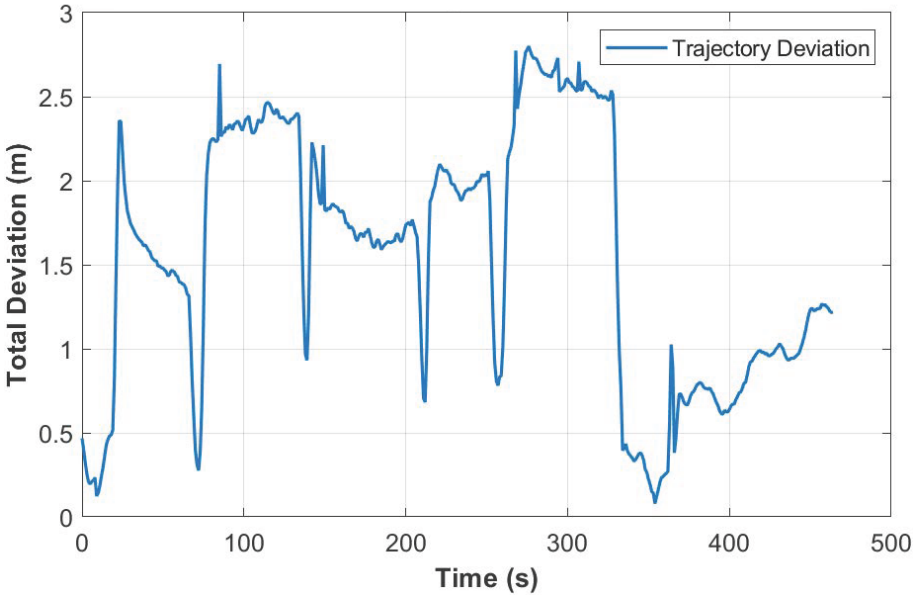


Figure 1. Temporal evolution of spatial deviation between identical flight plans executed under nominal and wind-affected conditions in the PX4 SITL environment [10].

3.2. Airspace Discretisation and Cell Representation

As mentioned in the previous paragraphs, the trajectory generator provides a realistic, dynamic representation of UAS motion that accounts for a wide range of interfering factors.

However, the European Regulation and the ASTM F3548-21 Standard establish that the airspace representation should be conceptualised as a grid, and that both flight trajectories and restricted volumes (e.g. Geozones, obstacles, etc.) are mapped into those grid cells. These cells consist of 4D volumes, which are 3D volumes extended with a temporal dimension, defined by start and end times. The start and end times account for the time span during which the flight is located within one cell [7].

To enhance computational efficiency, we have adopted an Octree spatial partitioning to discretise the airspace into adaptive cells. This hierarchical data structure recursively subdivides the 3D space into eight octants (children nodes). As illustrated in Figure 2, only the octants containing relevant information are further refined, while empty regions remain as larger, undivided cells. In this example, the refinement process focuses on the yellow sub-cells, representing the maximum subdivision resolution of the structure, whereas the blue and red nodes represent intermediate levels of the hierarchy.

Therefore, as shown in Figure 2, the action of finding a cell is performed by descending the tree from the root to a cell, with a depth d bounded by the refinement level. For a balanced tree, this yields a logarithmic access cost,

$$T_{\text{loc}} = \mathcal{O}(d) = \mathcal{O}(\log N), \tag{1}$$

where N denotes the number of cells. Since the computational cost of a conventional partitioning is,

$$T_{\text{loc}} = \mathcal{O}(N), \tag{2}$$

the computational difference between a conventional and an Octree partitioning is higher as the resolution increases, i.e., as the number of the cells increases.

Building on the Octree-based airspace representation, Sandra Amarillo et al. proposed a dedicated approach to strategic conflict detection in Proposal of UAS Strategic Conflict Detection Concept with a Centralised Service in Multi-USSP Environment Using an Octree Data Structure [8]. In that work, the authors proposed an implementation of the Search for Overlap Service (SFOS), a supporting microservice within the Flight Authorisation Service (FAS) framework. By leveraging the recursive subdivision of the Octree, the SFOS efficiently constructs an airspace representation and maps four-dimensional operational intents onto discrete three-dimensional grid cells to detect spatial overlaps, while temporal overlap is incorporated as an additional dimension of the cell representation. This approach not only facilitates interoperability in multi-USSP environments by allowing different providers to declare conflicts based on shared information (according to Standard ASTM F3548-21 [7]), but also leverages parallel computing to significantly accelerate the deconfliction process as traffic density increases.

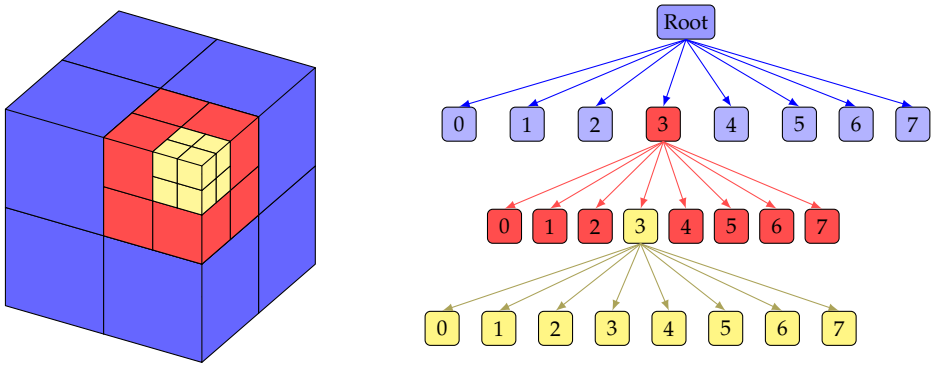


Figure 2. Octree structure representation. Blue indicates root cell; red indicates son of root cell; yellow indicates son of son of root cell (which is the maximum level of division).

3.3. UAS Geographical Zones

The UAS Geographical Zones, commonly referred to as Geozones, represent a fundamental tool for airspace management within the U-space ecosystem. According to Commission Implementing Regulation (EU) 2019/947 [31], these are defined as portions of airspace where UAS operations are facilitated, restricted, or excluded to address risks related to safety, security, privacy, or environmental protection.

From a dynamic management perspective, as outlined in the EASA Easy Access Rules for U-space [6], Geozones are not merely static constraints but can be activated temporarily to accommodate specific aerial events or emergency situations. An illustrative example occurs during an aerial exhibition: a geozone is defined by the horizontal limits of the event and extends vertically from the surface to the upper limit of the U-space volume. This temporary segregation effectively mitigates the risk of Mid-Air Collisions (MAC) by preventing unauthorised UAS from entering the high-risk volume.

In terms of computational modelling, representing these volumes accurately is critical for strategic deconfliction. While standard flight operational intents usually occupy one-dimensional lines (in terms of cells), Geozones frequently take the form of large, semi-static polyhedral volumes. As explained in Section 4.1, the geometric disparity between standard flight paths and large Geozones makes the latter the most computationally demanding case, necessitating a specialised Reinforcement Learning (RL) approach. By leveraging an Octree-based spatial discretisation, the proposed model efficiently exploits sparsely occupied regions within these large volumetric spaces.

3.4. Conflict Detection Workflow

The U-space standardisation framework laid down in Commision Implementing Regulation (UE) 2021/664 [5] and further developed in the EASA Easy Access Rules for U-space [6] defines several services that must be provided and used to operate UAS within U-space airspaces. One of these is the Flight Authorisation Service (FAS), which plays a critical role in flight permission management by ensuring that every 4D trajectory does not intersect with other occupied volumes or geozones, either spatially or temporally. In accordance with the regulatory framework, the provision of alternative routes upon conflict identification, as addressed in this work, constitutes an optional feature of the FAS [5].

As shown in Figure 3, the UAS operator submits a flight authorisation request containing a 4D trajectory to the FAS. The service checks whether the requested trajectory exhibits any spatio-temporal overlap with cells associated with previously accepted requests. If no such overlap is detected, the request is accepted. Otherwise, it is rejected unless the operation has a higher priority than an existing conflicting request, in which case the previous one is withdrawn. Upon rejection or withdrawal of a flight authorisation, the DRL-based strategic conflict resolution microservice proposed in this paper is invoked to generate an alternative route that avoids occupied cells while optimising distance and energy consumption in the 3D formulation.

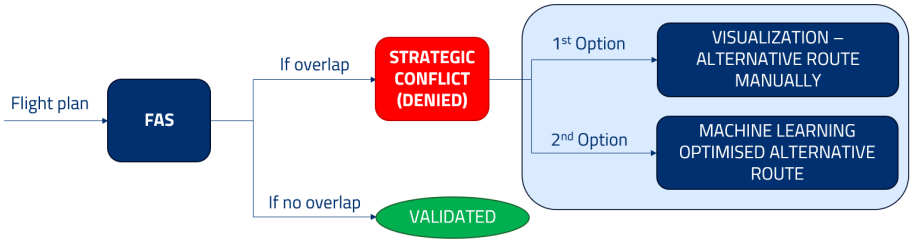


Figure 3. Workflow scheme.

The DRL-based strategic conflict resolution microservice is notified by the FAS, which serves as the triggering entity in the workflow. The microservice retrieves the corresponding 4D trajectory of the rejected or withdrawn flight and the set of occupied cells from the Octree representation. These cells, defined in latitude–longitude–altitude (LLA) coordinates, are discretised into an indexed grid structure (i,j), extended to (i,j,k) in the 3D case, to facilitate efficient state representation for DRL-based optimisation.

3.5. Energy Model

An approximate energy model has been developed to estimate the relative energy efficiency of the 3D planner. This model is also embedded inside the Reward Function of the RL agent so that, besides reaching the goal conflict-free, it explicitly optimises energetic cost. Note that the 2D model does not include this term, since its action space is restricted to constant-altitude motion and therefore excludes the most energy-demanding behaviours associated with vertical transitions.

The purpose of the model is to assign a *normalised* dimensionless energetic penalty to each discrete manoeuvre. Costs are defined relative to a straight, level step (cost equal to 1), and are derived by combining power and energy-per-distance figures reported by Gong, H. et al. in Modelling Power Consumptions for Multi-rotor UAVs [32], combined with the results from the work of Jacewicz, M. et al. in Quadrotor model for energy consumption analysis [33]. Here, energy consumption in discrete manoeuvres is derived from continuous motion analysis. Provided that detailed manoeuvre-resolved consumption data are scarce, the model is a first-order approximation and assumes a multirotor (quadrotor) platform. This assumption is supported by the limited data available, such as those reported by the

Italian National Civil Aviation Authority (ENAC), which indicate that at least 85% of UAS operators in Italy use quadrotors [34].

In addition to the quadrotor assumption, the following simplifications are adopted: wind and aerodynamic interactions are neglected; battery mass is assumed constant; acceleration effects are considered only for turning and vertical transitions; all manoeuvres are assumed feasible; and hover or fixed-wing dynamics are not modelled.

The resulting normalised costs (dimensionless, relative to a straight segment) are reported in Table 1. As expected, pure vertical ascent has a comparatively low cost due to the limited altitude increment per cell, whereas compound actions involving simultaneous turning and climbing (e.g., double ascending 180° turns) yield the largest penalties.

Table 1. Final normalized energy costs for all Maneuvers.

Section	Maneuver	Cost	Maneuver	Cost
Basics	Straight segment	1.0000	Ascent moving [1]	1.0012
	Vertical ascent	0.0488	Double ascent moving [2]	2.0024
	Vertical descent	0.0323	Descent moving	1.0005
	Double descent moving	2.0010		
Diagonal Movements	Straight segment	1.4142	Ascent	1.4151
	Double ascent	2.8302	Descent	1.4146
	Double descent	2.8292		
45° turns	Horizontal	1.4616	Descending turn	1.3467
	Double descending	2.6934	Ascending turn	1.3474
	Double ascending	2.6948		
90° turns	Horizontal	1.9233	Descending turn	1.6931
	Double descending	3.3862	Ascending turn	1.6939
	Double ascending	3.3878		
135° turns	Horizontal	2.8849	Descending turn	2.5396
	Double descending	5.0792	Ascending turn	2.5408
	Double ascending	5.0816		
180° turns	Horizontal	4.3273	Descending turn	3.8094
	Double descending	7.6188	Ascending turn	3.8112
	Double ascending	7.6224		

[1] Moving: it is being done while moving horizontally as well. [2] Double ascent: case when the drone is going downwards in movement and starts going upwards in movement. It is simplified to ascent twice.

3.6. Reference Method 1: A Pathfinding*

A* is a classical graph-based pathfinding algorithm that extends Dijkstra's method through the use of a heuristic function, thereby improving search efficiency. Its mathematical formulation is presented in Equation (3) [35]:

$$f(n) = g(n) + h(n), \tag{3}$$

where

- $g(n)$ is the accumulated cost from the initial node to the current node n.
- $h(n)$ is the heuristic function. It estimates the remaining cost from the current node n to the goal. It should always underestimate the remaining distance; hence, the Euclidean distance is often used here.
- $f(n)$ is the total cost of the most efficient path from the initial point to the goal while travelling through n.

Under the conditions met in this work, this algorithm ensures the most efficient (absolute-optimal) path. It is commonly used for distance optimisation. However, the cost

functions can be modified to optimise the desired variables. In this work, both distance and energy are optimised, so the energy costs are included in the cost function.

However, these classical graph-based algorithms, despite their high precision, incur significant computational costs, especially for long routes and 3D environments. As shown in Equation (4), the computational complexity grows exponentially with route length in the worst case [35]:

$$\mathcal{O}(b^d), \tag{4}$$

where

- b is the branching factor (number of adjacent nodes to the current node), which is 26 in the 3D environment ($3^3 - 1$), and 8 in the 2D environment ($3^2 - 1$).
- d is the route length, which quickly becomes prohibitive for long trajectories.

This constitutes the main limitation of the algorithm. The objective of this work is to develop an RL model that achieves path quality comparable to A*, while reducing computational cost by a significant margin.

3.7. Reference Method 2: Jump Point Search Method

Jump Point Search (JPS) is an online graph-pruning optimisation of the A* algorithm, specifically engineered to accelerate pathfinding on uniform-cost grid maps. Mathematically, JPS shares the same fundamental evaluation as A* (see Equation (3)). This algorithm was first introduced in 2011 by Harabor, D., in Online Graph Pruning for Pathfinding on Grid Maps [16].

While both algorithms are complete and return an optimal path, JPS improves efficiency by exploiting grid symmetries. Instead of expanding every adjacent node, JPS employs a recursive pruning strategy that identifies "jump points"—nodes where the optimal path is forced to change direction due to the presence of an obstacle, also known as "forced neighbours" [16].

Among the set of all equivalent optimal 2D paths, the JPS logic prioritises those in which diagonal movements are executed as early as possible, effectively bypassing large regions of empty space that A* would otherwise evaluate point-by-point. This recursive successor selection significantly reduces the number of operations in the Priority Queue (Open List), thereby reducing computational cost [16]. The performance difference between A* and JPS becomes more pronounced in low-density obstacle environments, where pruning opportunities are maximised. In the limiting case of a fully obstructed grid, both algorithms converge in terms of computational effort due to the absence of exploitable pruning structures.

The JPS was created for pathfinding in 2D grids [16]. However, some recent research has shown some extensions to 3D grids [36]. Nevertheless, its implementation in this paper has been limited to 2D due to its complexity in 3D and to the nature of this paper, which is not to compare traditional pathfinding algorithms but to test alternative Machine Learning-based methods.

3.8. Proposed Alternative: RL Model

As an alternative, we use Deep Q-Learning (DQN), which is a form of Reinforcement Learning (RL) that combines Q-learning with deep neural networks (DNNs) to handle large state spaces [37,38].

The computational complexity characterises the computation time of the RL model, as shown in Equation (5), where d denotes the route distance, implying a linear growth of computation time with respect to d:

$$\mathcal{O}(d), \tag{5}$$

The models have been created with the stable-baselines3 library by OpenAI. These models are based on an Agent that learns through a training process and progressively improves its behaviour according to a reward function. From the observed state space, the agent estimates the expected reward and, within the action space, selects the action that is expected to maximise the reward signal, thereby further improving its policy (see Figure 4).

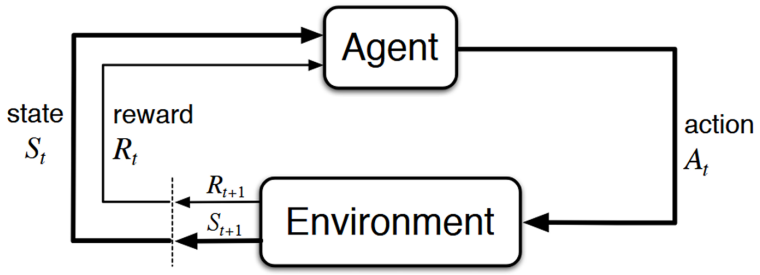


Figure 4. Reinforcement Learning model scheme [37].

There are many different techniques for training an RL model, as well as an extensive set of advanced architectures. However, all of them base their learning on the same idea. The Agent is the component that makes the decisions, based on the States it receives (the states can be understood as the senses of the agent), and the expected Reward from taking an Action. For instance, if the Agent is learning to play football, it will take an action, e.g., kicking the ball, and if the ball goes into the goal, it will see it in the States and receive a positive Reward. Otherwise, it will receive a negative Reward. This way, through many episodes of training, the Agent will learn to predict the Reward signal after taking an Action in a determined State, and consequently, it will learn to maximise the Reward. So, if it has been programmed correctly, this results in the Agent learning to play football [37].

3.8.1. Mathematical Formulation of the RL Approach

The routing problem is modelled as a Markov Decision Process (MDP), defined by the tuple $(\mathcal{S}, \mathcal{A}, p, R, \gamma)$, where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ the action space, $p(s', r \mid s, a)$ the transition probability, R the reward signal, and $\gamma \in [0, 1)$ the discount factor [37].

At each discrete time step t, the agent observes the state $S_t \in \mathcal{S}$, selects an action $A_t \in \mathcal{A}$ according to a policy $\pi(a|s)$, receives a reward R_{t+1}, and transitions to a new state S_{t+1}. The objective is to maximise the expected discounted return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}. \tag{6}$$

The optimal action-value function is defined as:

$$q^*(s, a) = \max_{\pi} q^{\pi}(s, a), \tag{7}$$

and satisfies the Bellman optimality equation:

$$q^*(s, a) = \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \max_{a'} q^*(s', a') \right]. \tag{8}$$

Given the dimensionality of the discretised 3D U-space grid, a tabular representation of $q(s, a)$ is computationally impractical. Therefore, a Deep Q-Network (DQN) is employed to approximate the optimal action-value function, such that $q(s, a; \theta) \approx q^*(s, a)$, where θ denotes the neural network parameters. The parameters are updated by minimising the temporal-difference loss:

$$\mathcal{L}(\theta) = \mathbb{E}_{(S_t, A_t, R_{t+1}, S_{t+1}) \sim \mathcal{D}} \left[\left(R_{t+1} + \gamma \max_{a'} q(S_{t+1}, a'; \theta^-) - q(S_t, A_t; \theta) \right)^2 \right], \quad (9)$$

where $\mathcal{D}$ denotes the replay buffer distribution and θ^- the parameters of a periodically updated target network.

In the proposed framework, S_t encodes the agent position within the discretised grid and its relation to the goal, while the transition structure is implicitly determined by the grid topology and occupied cells. The reward function embeds collision avoidance constraints and efficiency criteria (distance and energy cost), and γ regulates the balance between short-term manoeuvre penalties and long-term path optimality.

Unlike A*, which relies on an explicitly defined heuristic function $h(n)$, the RL agent learns an implicit approximation of the cost-to-go function through interaction with the environment. Once trained, policy inference scales linearly with the number of executed steps, which becomes advantageous in high-resolution 3D U-space environments.

Two different DQN models are proposed following an exhaustive evaluation and the identification of optimal RL hyperparameters and reward signal, which are key components of the model [28]. The details are provided in the following sections.

3.8.2. Generalisation, Underfitting and Overfitting Phenomena

An RL model learns by interacting with an environment, taking actions within a defined action space and receiving reward signals that guide its behaviour based on the observed state space. In other words, it can be viewed as a mapping from a dataset D_{set} of learning samples (x, y) into a predictive model $f(x|D_{set})$ [39].

In 2024, Aliferis et al., in Overfitting, Underfitting and General Model Overconfidence and Under-Performance Pitfalls and Best Practices in Machine Learning and AI [40], provided useful, intuitive explanations of these concepts. Figure 5 shows a classic demonstration of a regression of a continuous outcome Y given a continuous input X. Blue circles are training data, and white circles are unseen data.

In Figure 5a, the complex model (represented by the solid curve) fits the training data perfectly but fails to generalize to the unseen data. This phenomenon is known as overfitting: the model learns the noise in the training set rather than the underlying pattern. Conversely, the simpler model (dotted line) shows higher error on the training data but generalizes better to unseen data.

In Figure 5b, the complex model fits both the training and unseen data effectively, representing an optimal balance. Conversely, the simpler model (dotted line) fails to capture the data structure, performing poorly on both sets. This is known as underfitting.

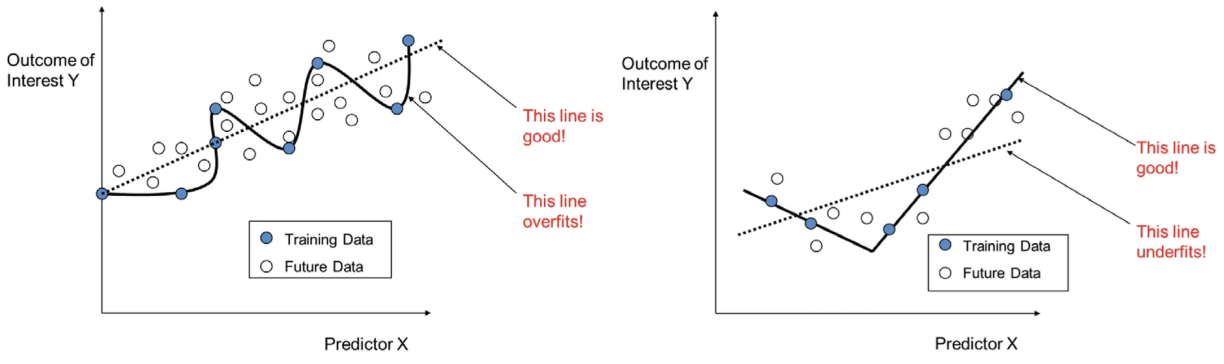


Figure 5. (**a**) Overfitting example. (**b**) Underfitting example [40,41].

When training an DRL model, excessive complexity often leads to overfitting, where the model memorises the training data but fails to generalise on unseen instances. On the other hand, an overly simplistic dataset can lead to underfitting, failing to capture the data underlying structure and yielding low predictive accuracy across datasets. These two phenomena, over- and underfitting, are directly related to the complexity and distribution of the training data.

4. Deep Reinforcement Learning Models

Two complementary DRL approaches are developed for optimal route generation. A 2D model is first designed and trained. As discussed in Section 1, 2D path planning is prioritised over 3D due to its lower computational and energy requirements. If no feasible solution is found in the 2D formulation, or if the mission inherently requires 3D motion, a dedicated 3D model is employed.

4.1. DQN Model—2D

As explained in Section 3.3, in scenarios where the flight operates at a constant altitude (as is the case in most operations), and the strategic conflict involves a Geozone rather than another flight, a 3D formulation is not required. In these cases, a 2D model is sufficient, as it reduces the dimensionality of both the state and action spaces compared to the 3D formulation. Furthermore, energy is not explicitly considered in this configuration, as satisfactory performance has been observed without incorporating this variable. This simplification reduces overall model complexity by limiting the number of states and actions.

4.1.1. State Space—2D

In this model, the state space is defined within the range $[-1, 1]$ and comprises the following components:

- Current Direction: 2 elements, each taking values in $\{-1, 0, 1\}$.
- Vision Vector: 24 elements encoding the occupancy of the two adjacent cells in each direction; a value of 0 indicates a free cell, while 1 denotes the presence of an obstacle.
- Relative Distance: 2 elements representing the normalised distance from the current position to the goal. Distances are scaled by the grid size, yielding values of -1 or 1 at maximum separation, 0 when the agent is located at the goal cell, and continuous values (32-bit precision) otherwise.

The above defines the state representation for a single time step. During training, a frame stacking technique is employed, whereby the states from the previous N time steps are concatenated to provide short-term temporal context and mitigate cyclic behaviours. In this work, the state vector includes the current state and the three preceding states. Consequently, the total state space dimension is

$$28 \frac{\text{states}}{\text{frame}} \times 4 \text{ frames} = 112 \text{ states}. \tag{10}$$

4.1.2. Reward Function—2D

The importance of carefully designing the reward function has been highlighted previously. In addition, prior research suggests that simpler reward structures can lead to superior performance. For instance, in 2017, AlphaGo Zero, introduced by Silver et al. [42], demonstrated that a minimal reward scheme—assigning +1 for a win and -1 for a loss—can outperform more complex formulations. Notably, this approach surpassed its predecessor, AlphaGo, which relied on engineered reward signals, achieving a 100–0 record in direct comparison. This result motivates the adoption of a sparse reward structure in the present navigation problem, where reducing reward shaping helps avoid unintended biases and

promotes more robust policy learning. Accordingly, the reward function in this work is defined as follows:

$$R = \begin{cases} R_{col}, & \text{if there is a collision} \\ R_{goal}, & \text{if goal achieved} \\ R_{dist} + R_{step}, & \text{otherwise} \end{cases} \tag{11}$$

where

- $R_{col} = -5$ is the terminal reward for collisioning,
- $R_{goal} = +5$ is the terminal reward for achieving the goal,
- $R_{dist} = d_{prev} - d_{new}$ is the distance reward, positive when the agent gets nearer to the goal and negative otherwise,
- $R_{step} = -0.1$ slight penalisation for each step taken.

4.1.3. Training Process—2D

This model has been trained using synthetic data generated from trajectories produced by the PX4 SITL simulator [10], as described in Section 3.1. In addition to flight trajectories, the training dataset also includes Geozones (see Section 3.3). The characteristics of both the training environments and realistic Geozones are illustrated in Figure 6. Figure 6a depicts a real scenario in which a planned trajectory is obstructed by three consecutive Geozones, similar to Scenario 2, which is later used for model evaluation in Section 5.1.2. In contrast, Figure 6b shows an example of a training grid used during the learning process. Notably, the training scenarios are intentionally more complex to promote robust performance under real-world conditions.

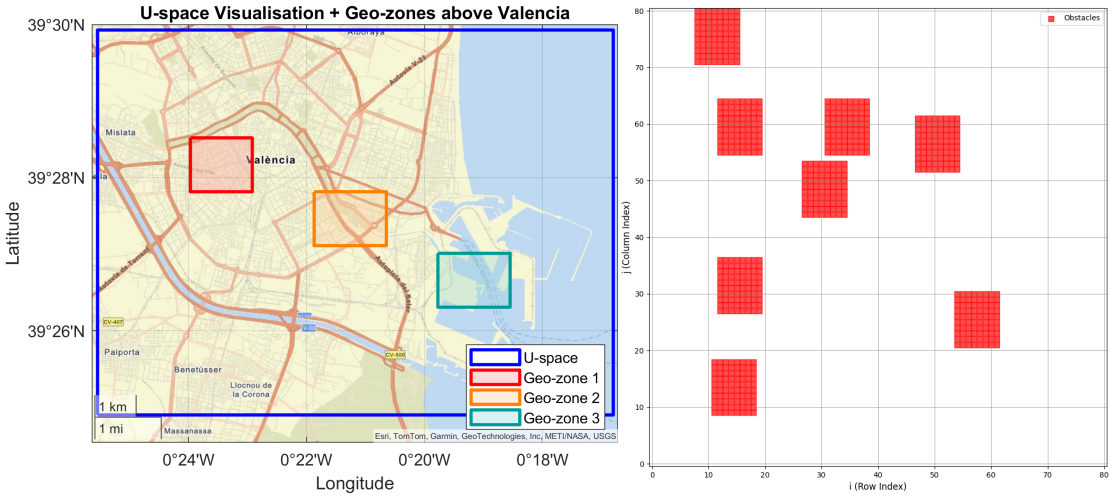


Figure 6. (**a**) Realistic geozones scenario. (**b**) Geozones of the training set.

Finally, the model was trained for approximately 25,000 episodes across 300 different grids of varying shapes, with randomly generated start and goal positions, thereby reducing the risk of overfitting.

4.1.4. DQN Hyperparameters Selection and Tuning—2D

The following hyperparameters have been chosen for the training of the model:

The performance of the Deep Q-Network (DQN) is highly sensitive to the configuration of its hyperparameters. In this work, the selected values (Table 2) were determined through iterative testing and a Learning Rate Test (LR-test). As established in the recent literature,

the Learning Rate (LR) is one of the most complex parameters to tune, as it is intrinsically coupled with the reward signal and the stability of the Q-value estimation [28].

Table 2. DQN Hyperparameters—2D model.

Hyperparameter	Value
Observation size	112
Action space size	8
Hidden layers	[128, 128]
Learning rate	1.0×10^{-4}
Discount factor (γ)	0.95
Replay buffer size	50,000
Batch size	256
Learning starts	100
Target network update interval	1000
Initial epsilon	1.0
Final epsilon	0.02
Exploration fraction	0.8
Gradient steps	1
Training frequency	Every 8 steps
Total training timesteps	1,000,000

A brief justification for the key parameters follows:

- Learning Rate (1.0×10^{-4}): A conservative value was chosen to ensure stable convergence and avoid catastrophic forgetting in complex obstacle configurations.
- Discount Factor ($\gamma = 0.95$): Balances the immediate need for collision avoidance with the long-term goal of reaching the destination.
- Hidden Layers [128, 128]: Provides sufficient representational capacity for the 112-state observation vector while maintaining millisecond-level inference latency.
- Exploration Fraction (0.8): Ensures the agent extensively explores the 2D grid environment before converging toward an optimal, deterministic policy.
- Batch Size (256): Reduces gradient variance, leading to more robust updates by averaging experiences from the replay buffer.

The combination of these parameters, refined through the LR-test, ensures that the agent achieves a high success rate while maintaining the near-optimal distance results presented in Section 5.

4.2. DQN Model—3D

4.2.1. State Space—3D

The state space of the 3D model is defined within the range $[-1, 1]$ and consists of the following components:

- Current Direction : 3 elements taking values in $\{-1, 0, 1\}$.
- Vision Vector: 26 elements. Compared to the 2D case, it encodes the occupancy of only the immediately adjacent cell in each direction.
- Relative Distance: 3 elements representing the normalised distance to the goal in 3D space (i.e., with one additional spatial dimension compared to the 2D formulation).
- Euclidean Normalised Distance: 1 scalar element.

In total, each training frame consists of 32 state variables. In this configuration, frame stacking is not employed, as the increased freedom of movement in the 3D environment reduces the likelihood of cyclic behaviours and local loops.

4.2.2. Reward Function—3D

The reward function is given by Equation (12):

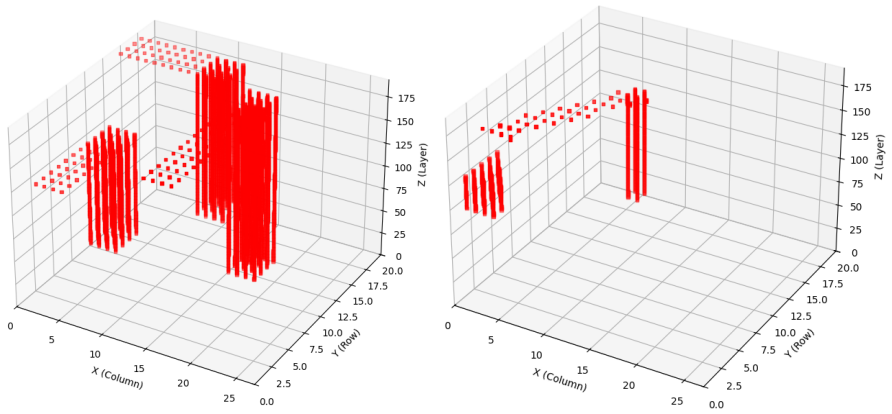
$$R = \alpha_{dist} \cdot R_{dist} + \alpha_{en} \cdot R_{en} + \alpha_{step} \cdot R_{step} + R_{term}, \quad (12)$$

where

- $R_{dist} = d_{prev} - d_{next}$ is the distance improvement (positive if it gets closer to the goal),
- R_{en} penalises energetically inefficient actions (according to the energy model—see Section 3.5),
- R_{step} is a fixed step cost,
- R_{term} is the terminal reward ($+10$ goal, -50 collision/out),
- $\alpha_{dist}, \alpha_{en}, \alpha_{step}$ are the corresponding weights, adjusted accurately.

4.2.3. Training Process—3D

The model has been trained using synthetic data derived from realistic flight trajectories generated by the PX4 SITL simulator [10] described in Section 3.1. These trajectories are used as a reference to construct additional pseudo-random trajectories with similar geometric shape and structural properties. The similarity between simulator-generated and pseudo-random trajectories is illustrated in Figure 7. The pseudo-random generation process is designed to promote generalisation of the Reinforcement Learning model [43]. As shown in the figure, the generated trajectories preserve the same overall structure as the original ones, while exhibiting a higher obstacle density. This design choice is intentional, as increased environmental complexity during training encourages robustness; consequently, satisfactory performance on the more challenging pseudo-random trajectories in Figure 7b is expected to transfer to, or improve performance on, the original simulator trajectories in Figure 7a.



(**a**) (**b**)

Figure 7. (**a**) Pseudo-randomly generated trajectories. (**b**) PX4 Simulator trajectories.

The training process has been carried out through 100,000 episodes, throughout 100 different scenarios, with start and end points randomly generated at the beginning of each episode, again ensuring the generalisation of the model.

4.2.4. DQN Hyperparameters Selection and Tuning—3D

The 3D model has been trained using the following hyperparameters:

The 3D environment significantly increases the state-space complexity and action density (26 possible moves). Consequently, the hyperparameters for the 3D DQN model (Table 3) were optimised to handle higher dimensionality and ensure convergence over a much larger training horizon ($50 \times \text{timesteps}_{2D}$). These values were further refined through an LR-test to balance stability and performance.

Table 3. DQN Hyperparameters—3D model.

Hyperparameter	Value
Observation size	128
Action space size	26
Hidden layers	[128, 256, 128]
Learning rate	4.13×10^{-5}
Discount factor (γ)	0.9008
Replay buffer size	50,000
Batch size	256
Learning starts	100
Target network update interval	1000
Initial epsilon	1.0
Final epsilon	0.02
Exploration fraction	0.8
Gradient steps	1
Total training timesteps	50,000,000

The rationale for the key parameters in the 3D model is as follows:

- Neural Network Architecture [128, 256, 128]: A deeper and wider structure was implemented to better approximate the non-linear Q-value functions required to coordinate vertical and horizontal navigation simultaneously. Although the size of one state is similar, the action space is much wider, increasing from 8 (2D) to 26 (3D).
- Learning Rate (4.13×10^{-5}): A lower LR than the 2D model was required to mitigate the high variance in gradients caused by the increased action space and the complexity of 3D obstacle avoidance.
- Discount Factor ($\gamma = 0.9008$): A similar value to the 2D for the same reason.
- Total Training Timesteps (50×10^6): Due to the "curse of dimensionality" in 3D grids, the agent required a significantly higher number of interactions to generalise a robust policy across heterogeneous scenarios.
- Observation size (128): Expanded to incorporate additional 3D spatial features and maintain sufficient context for decision-making.

4.3. Models Limitations—Partial Observability and Local Minima

As shown in Appendix A, despite achieving a high training success rate, the model does not reach 100% performance. This limitation is not due to insufficient training, but rather to the inherent characteristics of the state representation and reward design.

As described in Sections 4.1 and 4.2, the agent operates under partial observability, as its perception of the environment is restricted to a local neighbourhood of cells (5×5 grid in 2D and $3 \times 3 \times 3$ grid in 3D). In this context, the problem can be formally modelled as a Partially Observable Markov Decision Process (POMDP) [37,44].

Although frame stacking is employed in the 2D case, the information available to the agent remains purely local and does not provide sufficient global context of the environment.

In certain scenarios, reaching the goal requires temporarily increasing the distance to the target in order to bypass obstacles. However, the reward function is primarily driven by distance improvement, such that:

$$R_{dist} = d_{prev} - d_{new} \tag{13}$$

Under this formulation, actions that increase the distance to the goal yield negative rewards. Consequently, the agent tends to avoid such actions, even when they are necessary to successfully reach the goal.

This combination of local perception and reward shaping can lead the agent to become trapped in local minima, i.e., a situation in which all immediately available actions appear unfavourable, despite the existence of a better solution outside the agent's limited observation window [44].

An example of this behaviour is illustrated in Figure 8, where the agent oscillates and becomes stuck due to its inability to infer a feasible path beyond its limited observation window. The figure depicts a sequence of movements together with the corresponding distance-based reward evolution on a grid.

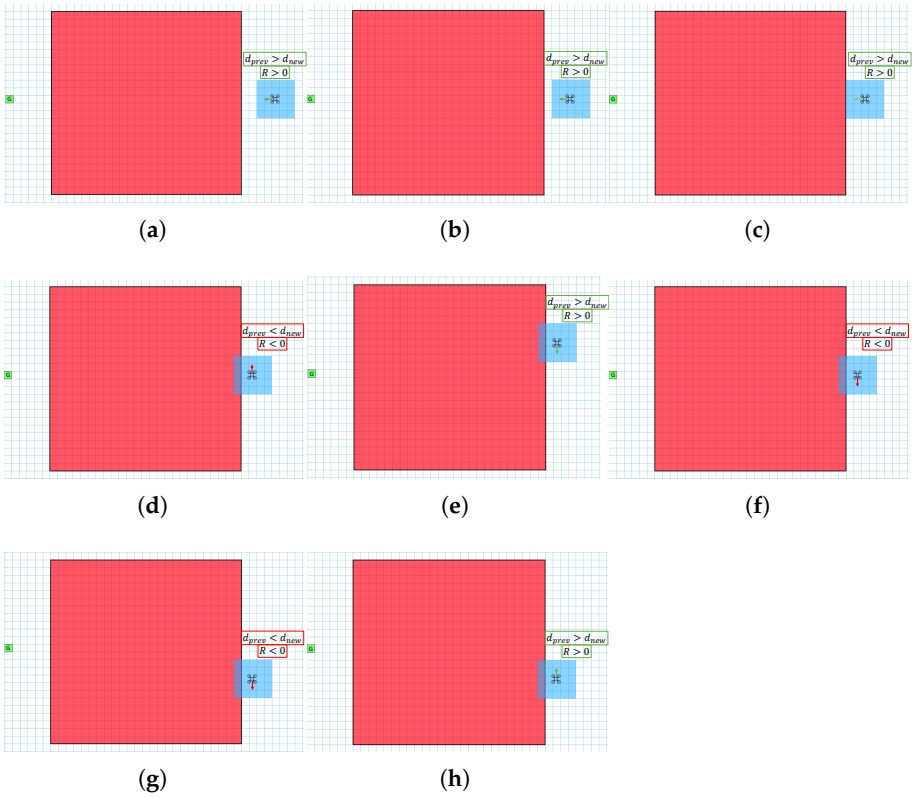


Figure 8. (**a**) Frame 1. (**b**) Frame 2. (**c**) Frame 3. (**d**) Frame 4. (**e**) Frame 5. (**f**) Frame 6. (**g**) Frame 7. (**h**) Frame 8.

In Figure 8, the agent is represented as a drone. The blue-shaded region corresponds to the portion of the grid observable within the current state space, the red region represents an obstacle that must be avoided, and the green cell denotes the goal (G). The sequence is described as follows:

(a) The agent does not detect any obstacle and moves towards the goal direction to reduce the distance and obtain a positive reward.
(b) The agent observes the same local environment as in (a).
(c) The agent continues to observe the same local configuration as in (a) and (b).
(d) The agent detects an obstacle within its observation space. Based on prior experience, it anticipates that moving forward may lead to a collision and a large negative reward. It therefore attempts an upward movement, despite the expected immediate penalty, as the associated cost is lower than collision.
(e) After four upward movements, the agent reaches a position where no apparent obstacle-free path towards the goal is available within its observation window. Continuing upwards is still expected to yield a negative reward, so it attempts the opposite direction by moving downward.

(f) The agent moves downward, increasing its distance to the goal and receiving a negative reward.
(g) After several downward movements, continuing in the same direction is still associated with negative reward. Consequently, the agent avoids further downward motion and reverts its decision.
(h) At this point, the agent oscillates between upward and downward movements, as both directions are associated with expected negative rewards. The agent returns to previously visited states, while earlier observations are no longer fully retained despite frame stacking. This behaviour leads to a loop induced by partial observability (POMDP) and limited memory of past states.

5. Results

5.1. 2D Model

In this section, the 2D model described in Section 4.1 is evaluated not only in the presence of flights, but also under the inclusion of Geozones. These restricted regions extend over large areas in the horizontal plane. Importantly, none of these scenarios is included in the training dataset, allowing an assessment of the model's ability to generalise and avoid overfitting. Given that Geozones can represent highly restrictive airspace (e.g., areas containing military bases), a protection buffer of five cells (approximately 200–250 m) is applied around all Geozones (cells shown in light blue in next figures). The following subsections present the evaluation results of the 2D model across three different scenarios.

5.1.1. Scenario 1—2D

The first scenario, shown in Figures 9 and 10, presents a strategic conflict between a flight request and two Geozones (including the five-cell safety buffer surrounding them) that restrict operations within the affected area.

In this scenario, as shown in Table 4, the ML model achieves a 98.2% optimality in terms of distance. While JPS is significantly faster than A* (0.0692 s vs. 0.1705 s), the DRL model still provides the best performance in terms of latency, reducing JPS computation time by a factor of $\times$4.55. On this relatively short route, the absolute time difference between JPS and ML is in the order of centiseconds; however, the DRL's near-instantaneous inference remains superior for high-frequency activation requests, allowing it to resolve more than four strategic conflicts by the time JPS could resolve only one. Then, simplicity has been shown to be extremely efficient in terms of the reward function, as predicted in Section 4.1.2.

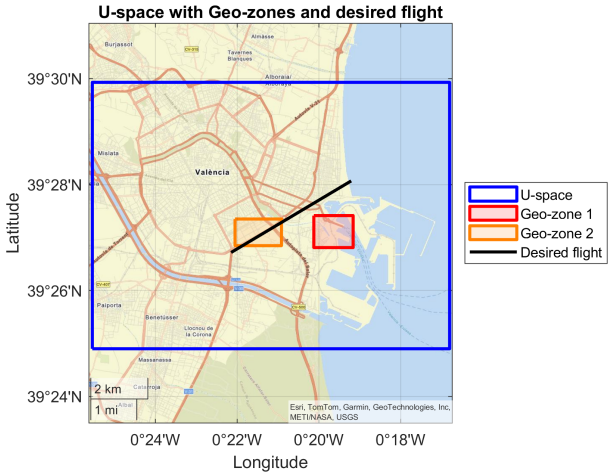


Figure 9. Representation of the flight request that is in conflict with two Geozones when it is about to cross a part of the city of Valencia.

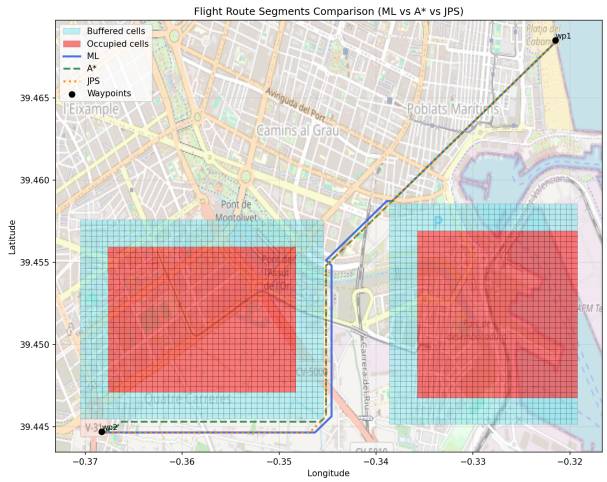


Figure 10. Evaluation of the 2D model in the first scenario

Table 4. Numerical results of evaluation in the first scenario—2D.

Method	Length [Steps]	Time [s]
A*	112	0.1705
JPS	112	0.0692
ML (DRL)	114	0.0152
Difference (ML vs. JPS)	1.8%	×4.55

5.1.2. Scenario 2—2D

This scenario represents a city-scale conflict in the U-space airspace over Valencia. Figures 11 and 12 shows that the planned flight trajectory overlaps with three different Geozones, as it traverses the entire city of Valencia.

In this complex city-scale scenario involving three different Geozones, the computational advantages of the DRL model become even more evident. As shown in Table 5, the DRL approach reduces JPS response time by a factor of ×7.83, while maintaining a distance optimality of 99.57%. JPS performs as a strong benchmark in this high-density environment, outperforming A* by more than a factor of two. Nevertheless, for the strategic resolution of multiple simultaneous requests, the millisecond-level latency of the DRL model makes it a more scalable solution. In practical terms, this implies that, under equivalent conditions, the DRL model can resolve approximately eight strategic conflicts in the time required by JPS to solve a single one, while preserving near-optimal path quality.

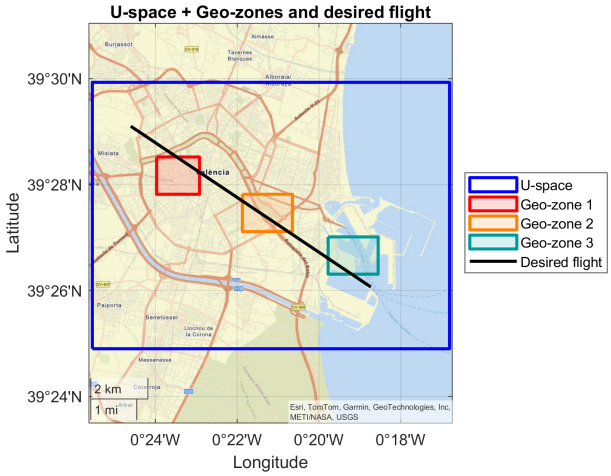


Figure 11. Representation of the flight request that is in conflict with three Geozones as it passes through the entire city of Valencia.

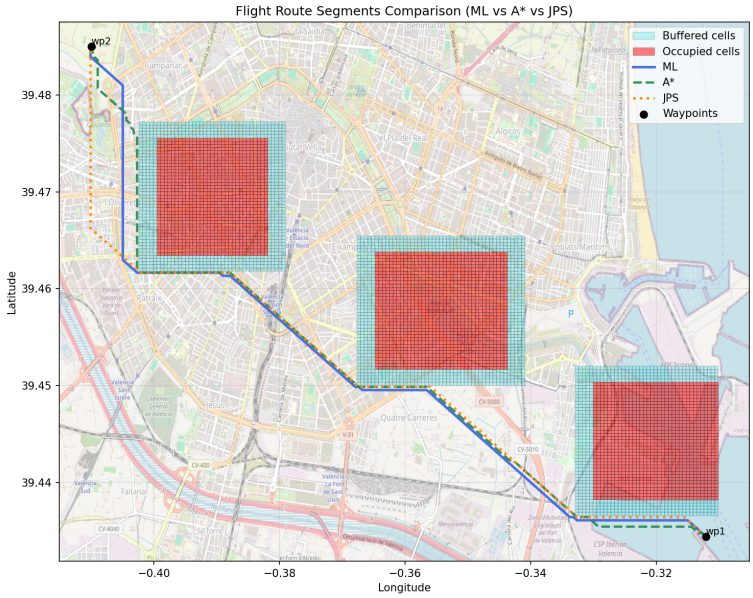


Figure 12. Evaluation of the model in the second scenario—2D.

Table 5. Numerical results of evaluation in the second scenario—2D.

Method	Length [Steps]	Time [s]
A*	230	0.5532
JPS	230	0.2420
ML (DRL)	231	0.0309
Difference (ML vs. JPS)	0.43%	×7.83

5.1.3. Scenario 3—2D

In this scenario, the 2D model is evaluated in the presence of conflicting flight requests. The DRL model generates a flight plan consisting of four waypoints (three segments). As shown in Figure 13, the new flight request cannot be approved, as it conflicts with the operational volumes of previously accepted flights.

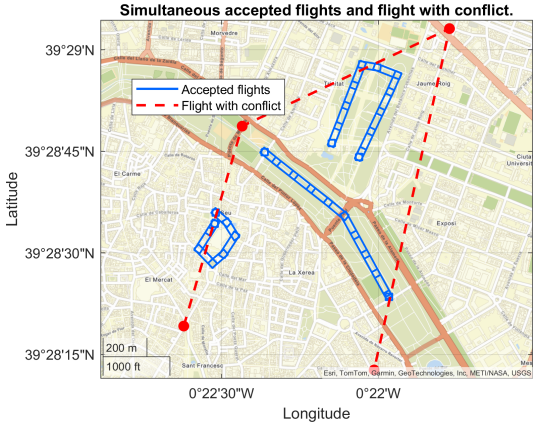


Figure 13. Representation of the operational volumes of the previously accepted flights (blue volumes) and the flight request that is in conflict (red line).

Figure 14 demonstrates a successful and smooth resolution, where each segment of the trajectory avoids the occupied volumes. As shown in the numerical results (Table 6), the DRL model achieves a 96.4% distance optimality compared to deterministic benchmarks. While JPS provides a strong deterministic baseline, outperforming A* by nearly 40% in

latency, the DRL model remains the most efficient solution in terms of computation time, requiring only 0.0128 s per query. This corresponds to a $\times 3.76$ speedup over JPS.

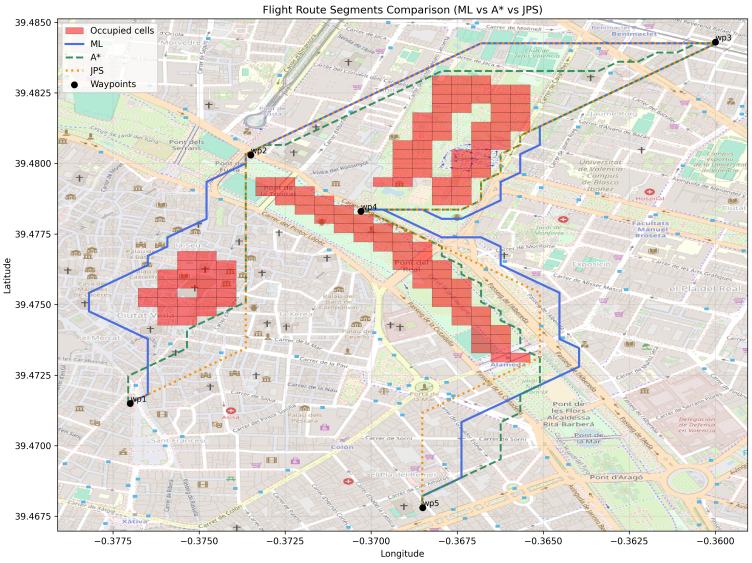


Figure 14. Evaluation of the model in the third scenario—2D.

Table 6. Numerical results of evaluation in the third scenario—2D.

Method	Length [Steps]	Time [s]
A*	110	0.0820
JPS	110	0.0482
ML (DRL)	114	0.0128
Difference	3.6%	$\times 3.76$

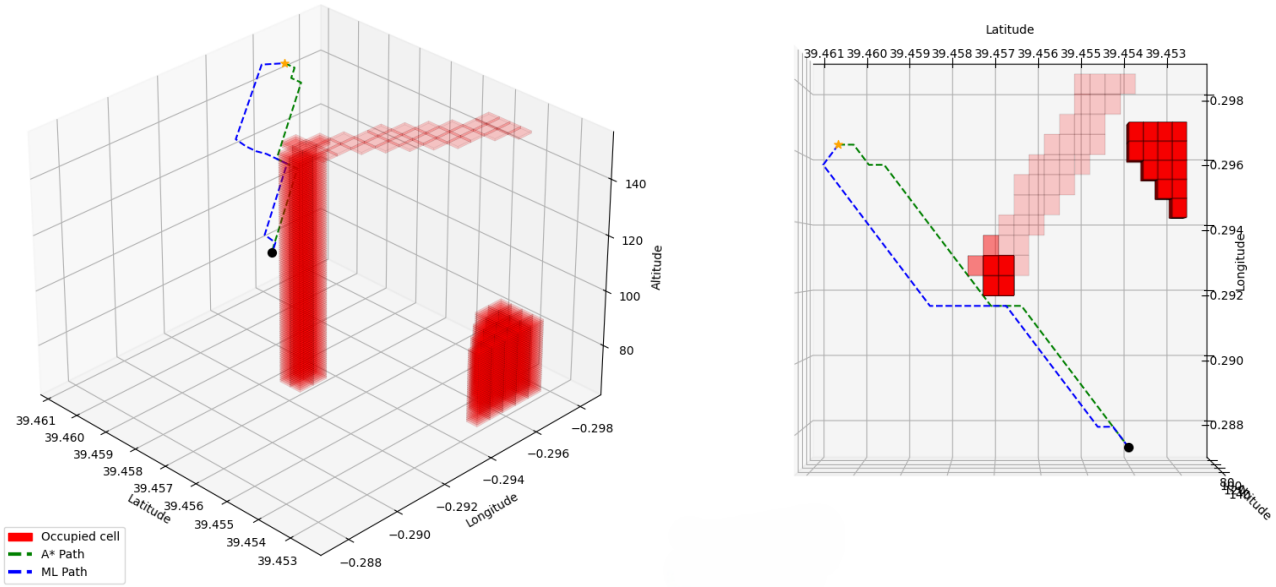
In this context, the JPS algorithm serves as a competitive deterministic reference; however, the DRL inference-based approach enables the rapid response required in time-critical scenarios, such as emergency activations, where minimising large-scale replanning is essential. If the flight request considered in this scenario were classified as a priority operation, three existing flight plans would need to be revoked to accommodate it, highlighting the operational impact of the proposed conflict-resolution tool in dense U-space environments. By generating a conflict-free alternative in real time, the DRL-based tool reduces the need for such withdrawals and supports improved operational throughput in U-space environments.

5.2. 3D Model

The 3D model described in Section 4.2 has been evaluated in different environments generated using the SITL PX4 simulator described in Section 3.1 [10]. Importantly, none of these environments were included in the training dataset, ensuring that the agent had no prior exposure to them. This evaluation protocol enables the assessment of generalisation capabilities rather than memorisation, providing a realistic indication of the model's performance. Note that the 3D scenarios used in this work are derived from the scenario presented by Amarillo, S. et al. in Proposal of UAS Strategic Conflict Detection concept with a centralised service in multi-USSP environment using an Octree data structure [8], where a U-space environment over the city of Valencia was simulated with 10 concurrent flights.

5.2.1. Scenario 1—3D

The results for the first scenario are shown in Figure 15. The blue line represents the A* path, while the green line corresponds to the DRL-generated path. Both approaches successfully reach the goal while avoiding the occupied (red) cells, exhibiting visually similar trajectories. However, the numerical results, presented in Table 7, indicate that path length and energy consumption increase by 15% and 6.51%, respectively, while computation time is reduced by approximately 50%. These results can be considered near-optimal, as the increase in cost metrics remains limited while computational efficiency is significantly improved.



(**a**) (**b**)

Figure 15. Evaluation of the 3D model in the first scenario: (**a**) isometric view; (**b**) top view.

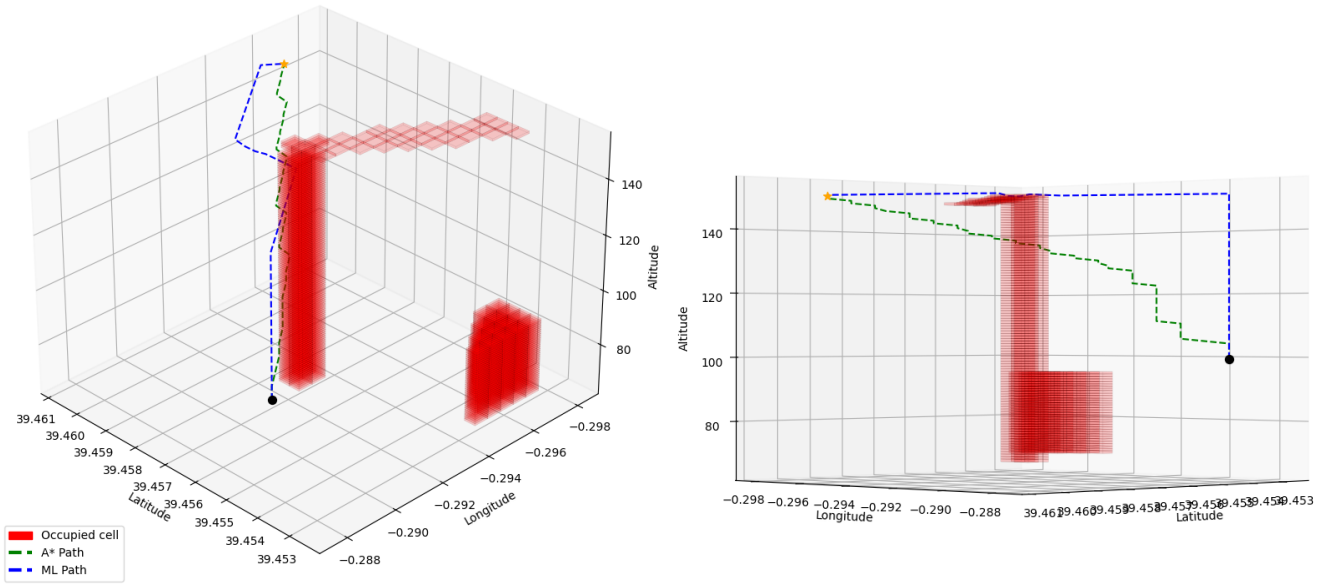
Table 7. Numerical results of evaluation in the first scenario—3D.

Method	Length [Steps]	Energy [-]	Time [s]
A*	20	26.69	0.023
ML (DRL)	23	28.43	0.011
Increment	15.00%	6.51%	×2.091

Figure 16 presents the results of a second simulation within the same scenario, considering a high-priority flight authorisation request with a different take-off location. In the horizontal plane, both methods exhibit similar behaviour, while the main differences arise in the vertical dimension due to the altitude variation between the start and goal positions. As shown in Table 8, the DRL route is longer (+32%) and slightly less energy-efficient (+12%). However, the key result lies in the computation time: A* requires 35.47 s, whereas the DRL approach obtains a solution in only 0.038 s, corresponding to a reduction of three orders of magnitude. For longer trajectories, A* exhibits exponential growth in computational cost (see Equation (4)), whereas the DRL inference time increases only mildly (see Equation (5)), making the difference increasingly significant in large-scale scenarios.

The increased path length in this 3D scenario can be attributed to the high density of altitude changes (representing up to 70% of the mission), which causes the agent's local perception to favour conservative vertical transitions over the globally optimal geometric path provided by A^*. This phenomenon is reflected in the structure of the agent's state space. As detailed in Section 4.2.1, the dominant state component guiding the agent towards the goal

is the Relative Distance. In this scenario, the scale of the vertical dimension (Z) significantly exceeds that of the horizontal components (X and Y). As a result, the value function initially prioritises the reduction of the vertical error, effectively guiding the agent towards the target altitude before refining the planar position. Once the vertical component becomes negligible (i.e., the agent reaches the target altitude layer), the horizontal components regain relative importance in the state representation, triggering the final convergence in the X and Y directions.



(**a**) (**b**)

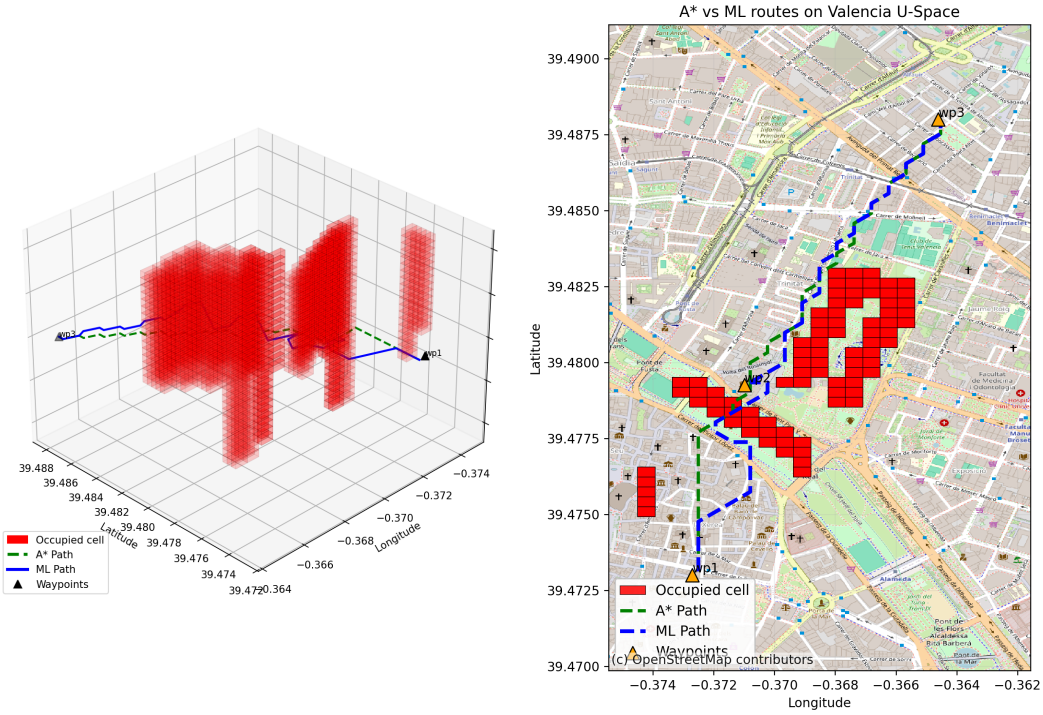
Figure 16. Evaluation of the 3D model in the first scenario with different altitudes: (**a**) isometric view; (**b**) lateral view.

Table 8. Numerical results of evaluation in the first scenario (special case of changing altitudes)—3D.

Method	Length [Steps]	Energy [-]	Time [s]
A*	66	27.93	35.47
ML (DRL)	87	31.29	0.038
Increment	32.00%	12.01%	×932.42

5.2.2. Scenario 2—3D

In the next scenario, the flight plan consists of three waypoints. Both segments of the route are in conflict with a previously accepted flight, requiring two distinct resolution actions. The resulting alternative trajectories are shown in Figure 17. As illustrated in Figure 17b, in the first segment both the DRL and A* solutions pass below the previously accepted flights. In the second segment, both approaches avoid the existing trajectories by routing around them. A notable reduction of computation time is seen in Table 9, where the length and the energy of the route are quite similar to the A*.



(**a**) (**b**)

Figure 17. Evaluation of the 3D model in the second scenario: (**a**) isometric view; (**b**) top view overlaid on the scenario map.

Table 9. Numerical results of evaluation in the second scenario—3D.

Method	Length [Steps]	Energy [-]	Time [s]
A*	47	57.67	0.056
ML (DRL)	49	65.53	0.013
Increment	4.08%	13.63%	×4.31

6. Discussion

The results obtained across both 2D and 3D scenarios confirm that the proposed DRL-based framework is a computationally scalable alternative to classical graph-search methods for strategic conflict resolution in U-space environments.

From a performance standpoint, two distinct behaviours can be observed:

- In 2D scenarios, the DRL model achieves consistently high performance (98.2% in Scenario 1, 99.57% in Scenario 2 and 96.4% in Scenario 3) while reducing computation time by factors between ×4.0 and ×8.0. Since the branching factor in a 2D environment is limited ($b = 8$), the exponential complexity of A* does not yet become as apparent as in 3D, at least for the approval stage. However, as mentioned in the Introduction (Section 1), at the activation stage, in emergency missions or when a Dynamic Airspace Reconfiguration is activated, the time required to compute a feasible route must be minimised. In these contexts, the proposed model is significantly more suitable than classical graph-search algorithms due to its substantially lower inference time.
- In 3D scenarios, an even more pronounced reduction in computation time is observed. In a specific case, a speedup exceeding ×900 is achieved. Although such extreme values are not representative of the average performance, they arise in configurations requiring significant altitude transitions and volumetric detours. In these cases, the search depth d increases substantially, while the branching factor in 3D ($b = 26$)

amplifies the combinatorial growth of A*. Although these scenarios are not predominant in routine cruise-level deconfliction, they may occur in complex operational cases involving vertical constraints. Therefore, while the $\times 900$ speedup should not be interpreted as a baseline improvement, it highlights the sensitivity of exponential graph-search methods to vertically complex environments. In contrast, the DRL inference time remains stable and largely independent of the number of required vertical manoeuvres.

Beyond the pure computational comparison, the relevance of these results must be interpreted in the operational context of U-space services. In the approval stage, where flight plans are typically submitted hours or even days in advance, the marginal reduction from, for instance, 0.25 s to 0.06 s may not significantly impact system-level performance. In such cases, A* remains a robust and fully deterministic alternative.

However, the activation stage imposes a distinct operational constraint. According to ASTM F3548-21 [7] and the EASA Easy Access Rules for U-space [6], activation responses must be delivered within strict temporal thresholds in most operational cases. In high-density environments, even moderate route lengths can lead to exponential growth in A* computation time due to its $\mathcal{O}(b^d)$ complexity. Although this effect is less pronounced in 2D ($b = 8$) than in 3D ($b = 26$), the branching factor still leads to significant scalability limitations as grid resolution increases or congestion intensifies.

Therefore, the advantage of the DRL model in the 2D scenario should not be interpreted merely as a speed improvement, but rather as the provision of bounded inference time. Once trained, the policy execution scales linearly with route length ($\mathcal{O}(d)$), independently of the obstacle configuration. This property is particularly valuable in emergency missions or Dynamic Airspace Reconfiguration (DAR) events, where multiple flights may need to be re-evaluated simultaneously and near-instantaneously. In such cases, deterministic but exponentially scaling algorithms may introduce cascading delays, thereby compromising overall airspace efficiency.

Furthermore, the 2D results indicate that a carefully designed reward function, even without explicit energy penalisation, can be sufficient to induce near-optimal path behaviour. This observation suggests that increased reward complexity is not necessarily beneficial and is consistent with findings in the literature, where reward shaping and hyperparameter tuning often play a more critical role than architectural sophistication.

The fact that the 2D model achieves $> 99\%$ optimality without explicit energy terms suggests that the distance-based formulation provides an implicit regularisation effect, discouraging unnecessary manoeuvres.

The inclusion of Jump Point Search (JPS) as a benchmark provides further insight into the trade-offs involved in U-space pathfinding. The results indicate that JPS is a highly effective method for long-range strategic planning, as it efficiently prunes the search space and mitigates the combinatorial growth associated with A*. For long trajectories, JPS exhibits significant performance gains, making it a viable deterministic candidate for the approval stage.

However, as U-space operations progress toward the activation phase or emergency prioritisation, the $\mathcal{O}(d)$ complexity of DRL inference provides a level of scalability that JPS cannot match. While JPS can approach DRL performance in short segments, its iterative nature remains dependent on obstacle topology, whereas DRL inference latency remains bounded and predictable, largely independent of environmental complexity.

Nevertheless, some limitations remain. Although the DRL approach consistently reaches the goal state in most cases, it does not provide a deterministic guarantee of optimality or completeness. In safety-critical, certified systems, this limitation must be

carefully addressed, potentially through hybrid architectures in which DRL generates rapid candidate solutions, while classical algorithms verify feasibility when time permits.

In summary, the 2D and 3D experiments confirm that while A* remains suitable for low-density or non-time-critical approval processes, the DRL approach provides a structurally more scalable solution for activation-stage conflict resolution, emergency rerouting and dynamically reconfigurable U-space environments. The advantage is not merely numerical but architectural: bounded response time becomes the key operational metric as traffic density increases.

7. Conclusions

This paper presented a three-stage framework for strategic conflict resolution in U-space environments, integrating Octree-based airspace discretisation, A* pathfinding and Deep Reinforcement Learning (DRL) models. The approach was designed to address the computational limitations of classical graph-search methods in increasingly dense and volumetric U-space scenarios.

The proposed DRL models were trained on realistic PX4 Software-In-The-Loop trajectories and evaluated on previously unseen scenarios, thereby ensuring generalisation. In 2D environments, the DRL planner achieved near-optimal (96–99.6% optimality) route lengths while consistently reducing computation time compared to A* and even JPS. While JPS proved to be an excellent and highly efficient deterministic benchmark—particularly for long routes where it shows a clear advantage over A*—the DRL architecture demonstrated superior performance for high-density, time-critical applications. We conclude that while deterministic models like JPS are ideal for standard planning, DRL-based algorithms provide the near-instantaneous response necessary for the most demanding U-space operational requirements.

In 3D configurations, where the branching factor and route depth significantly increase computational complexity, the DRL model maintained bounded inference times and avoided the exponential scaling behaviour inherent to classical search algorithms. In specific, vertically complex cases, substantial reductions in computation time were observed.

These results indicate that learning-based planners can provide a scalable alternative for activation-stage strategic deconfliction, as well as in scenarios that require near-instantaneous responses, such as emergency missions or Dynamic Airspace Reconfiguration (DAR). While A* remains a deterministic benchmark for optimality, the DRL approach offers predictable latency and operational feasibility as traffic density and volumetric complexity increase.

Overall, the findings support the integration of carefully designed DRL-based planning modules into future U-space service architectures, thereby contributing to the development of responsive and computationally efficient strategic conflict-resolution mechanisms.

8. Future Work

Although the proposed framework demonstrates promising scalability and computational advantages, further extensions are required to enhance its operational maturity and ensure alignment with regulatory requirements.

First, the current energy model is intentionally simplified and based on first-order approximations for multirotor platforms. Future work should incorporate higher-fidelity energy-estimation models derived from real flight data, including wind disturbances and vehicle-specific performance envelopes, as well as fixed-wing platforms. This would allow the reward formulation to better reflect operational constraints and platform diversity.

Second, the present study focuses on single-flight strategic resolution. In dense U-space environments, multiple simultaneous replanning events may occur, particularly

during Dynamic Airspace Reconfiguration (DAR) or emergency prioritisation. Extending the framework to support multi-agent or cooperative resolution strategies is a necessary step for large-scale deployment in multi-USSP environments. This may involve decentralised learning architectures or hybrid coordination mechanisms between deterministic and learning-based planners.

Third, although the DRL model demonstrates strong generalisation in unseen city-scale scenarios, further validation in larger-scale, higher-density simulations is required. In particular, heterogeneous traffic patterns and layered altitude structures should be incorporated to rigorously assess scalability and robustness.

Furthermore, the evolution of metaheuristic algorithms paves the way for new comparative analyses. Recently, Beetle Antennae Search (BAS) methods [17,18] have demonstrated notable efficiency in trajectory optimisation thanks to their low computational cost and ability to avoid local minima. Integrating BAS variants into discrete spaces would allow for an assessment of whether they can outperform the agility of ML (DRL) in the activation phase or, conversely, whether their iterative nature still imposes limitations on the direct execution of a previously trained neural policy.

Finally, from a certification and safety perspective, future research should explore hybrid verification schemes in which learning-based planners generate candidate routes that can be validated or constrained by deterministic safety filters. Such architectures could facilitate regulatory acceptance while preserving the computational benefits observed in this work.

These directions aim to progressively bridge the gap between experimental validation and operational integration of learning-based strategic conflict resolution services within future U-space ecosystems.

Author Contributions: Conceptualization, M.G.; methodology, M.G.; software, M.G., S.A.; validation, M.G., S.A. and A.S.; formal analysis, J.V.B., S.A., M.G.; investigation, M.G.; resources, M.G., S.A. and A.S.; data curation, M.G. and S.A.; writing—original draft preparation, M.G.; writing—review and editing, J.V.B. and S.A.; visualization, M.G.; supervision, J.V.B. and S.A.; project administration, J.V.B.; funding acquisition, J.V.B. All authors have read and agreed to the published version of the manuscript.

Funding: This research was conducted within the framework of the U-AGREE project. This project has received funding from the SESAR 3 Joint Undertaking (JU) under grant agreement No 101167187. The JU receives support from the European Union's Horizon Europe research and innovation programme and the SESAR 3 JU members other than the Union.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Data available on https://doi.org/10.5281/zenodo.18790882 (accessed on 20 May 2026). For further information, contact the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

AI	Artificial Intelligence
BADA	Base of Aircraft Data
BAS	Beetle Antennae Search
D3QN	Dueling Double Deep Q-Network
DAR	Dynamic Airspace Reconfiguration
DNN	Deep Neural Networks
DQN	Deep Q-Network

DRL	Deep Reinforcement Learning
EASA	European Union Aviation Safety Agency
ENAC	Ente Nazionale per l'Aviazione Civile
FAS	Flight Authorisation Service
FBAS	Fallback Beetle Antennae Search
JPS	Jump Point Search
LLA	Latitude, Longitude, Altitude
MAC	Mid-Air Collisions
MDP	Markov Decision Process
ML	Machine Learning
NN	Neural Networks
POMDP	Partially Observable Markov Decision Process
PPO	Proximal Policy Optimization
RL	Reinforcement Learning
SAC	Soft Actor-Critic
SFOS	Search for Overlap Service
SITL	Software-In-The-Loop
UAM	Urban Air Mobility
UAS	Unmanned Aircraft Systems
USS	UAS Service Supplier
USSP	U-space Service Provider
UTM	UAS Traffic Management

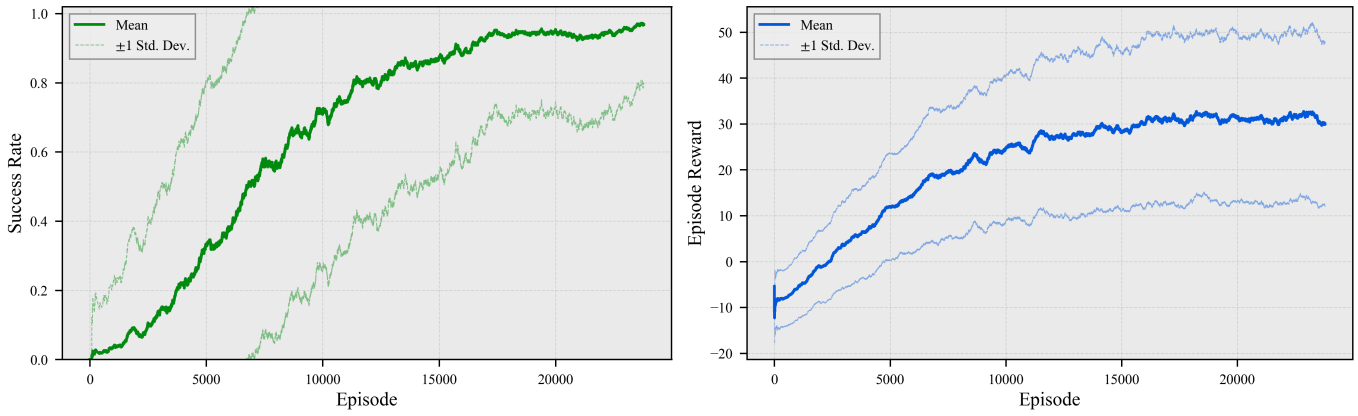
Appendix A. Training Results

The appendix contains the training results obtained for both models. The figures included in this section show the evolution of the models throughout the training episodes, illustrating how their performance improved from an initial state with no learned behaviour to a final stage in which they have learned an appropriate policy.

Appendix A.1. Training Results—2D Model

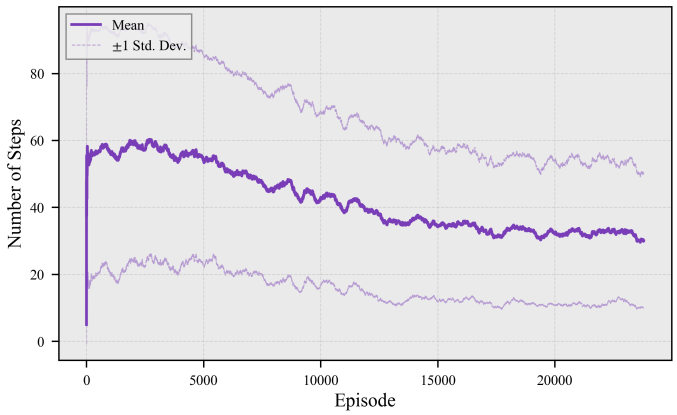
The training results of the 2D model are presented in Figure A1.

In Figure A1a, an increase in success rate from 0 to an average of approximately 95% is observed. Initially, the model operates without prior knowledge of the environment; however, it progressively learns to maximise expected reward, successfully reaching the goal in the vast majority of episodes. The remaining 5% of failures are further analysed in Section 4.3.



(**a**) (**b**)

Figure A1. *Cont.*



(**c**)

Figure A1. (**a**) **Success** rate 2D during training. (**b**) Mean of episode rewards 2D during training. (**c**) Number of steps 2D during training.

Figure A1b illustrates the mean reward obtained throughout training. An overall increasing trend is observed from the beginning of the learning process, indicating that the agent is effectively learning to optimise the reward function and, consequently, to reach the goal more efficiently.

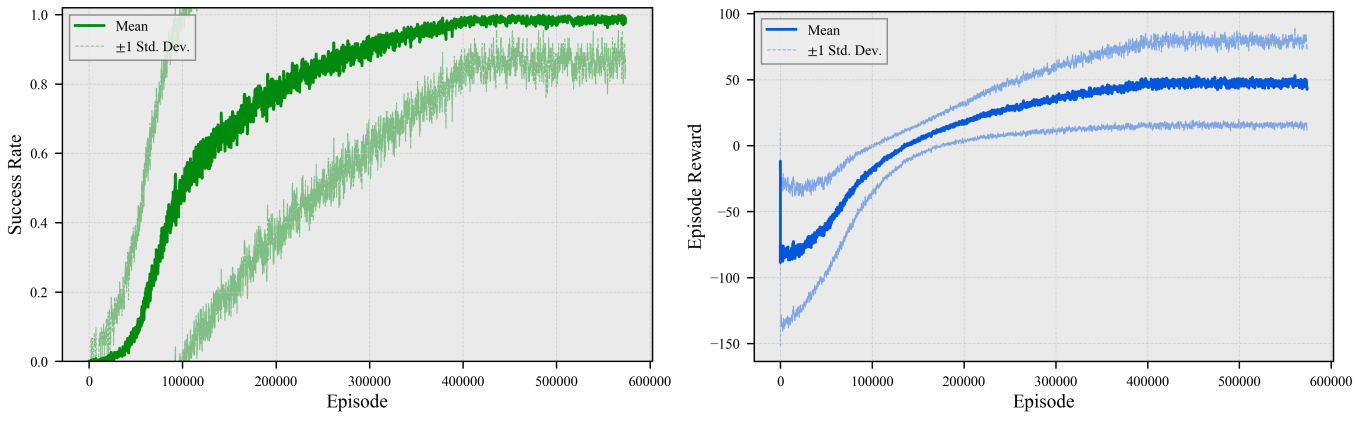
Finally, Figure A1c shows the number of steps per episode. The average decreases from approximately 60 to around 30 steps, indicating that the agent learns progressively shorter trajectories and thus exhibits increasingly efficient behaviour.

Individually, each metric provides only partial insight into the learning performance. For instance, a high success rate alone does not guarantee optimal behaviour, as it may still be associated with inefficient trajectories if the reward function is poorly shaped. However, when considered jointly, the three metrics provide consistent evidence that the learned policy is both stable and efficient.

Appendix A.2. Training Results—3D Model

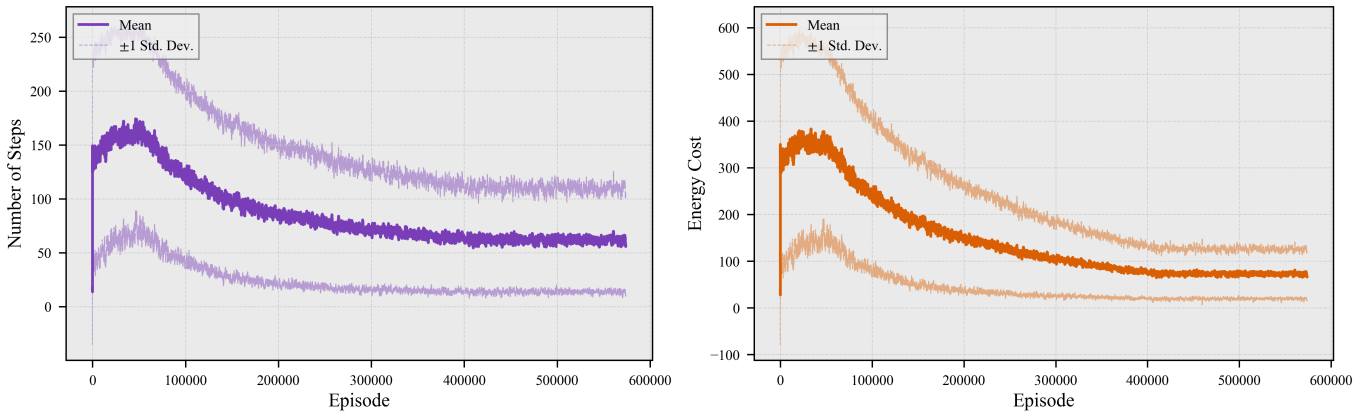
Figure A2 shows the results of the 3D model training process.

In Figure A2a, the success rate is illustrated, increasing from 0 at the beginning of training to approximately 95% at convergence, indicating that in 95% of episodes the agent successfully reaches the goal without collision. While this confirms that the agent learns to achieve the objective, it does not provide insight into the efficiency of the resulting behaviour. The remaining 5% of failures are analysed in Section 4.3.



(**a**) (**b**)

Figure A2. *Cont.*



(**c**) (**d**)

Figure A2. (**a**) **Success** rate 3D during training. (**b**) Mean of episode rewards 3D during training. (**c**) Number of steps 3D during training. (**d**) Energy costs 3D during training.

The episode rewards, presented in Figure A2b, increase from approximately −80 to around +50. The overall upward trend in cumulative reward indicates that the agent learns to maximise the reward function and, consequently, to jointly optimise distance and energy-related objectives, provided that the reward formulation correctly encodes these criteria.

Figure A2c reports a significant reduction in the number of steps per episode over the course of training, indicating that the agent progressively learns more efficient trajectories that reach the goal in fewer iterations. When considered together with the previous metrics, this suggests that the learned policy improves both in terms of success rate and efficiency.

Furthermore, Figure A2d exhibits a marked decrease in energy consumption throughout training, supporting the hypothesis that the agent adopts increasingly energy-efficient behaviour. The similarity between this trend and that observed in Figure A2c is expected, as energy consumption is directly correlated with the number of steps taken.

References

1. United Kingdom Government. A Brief History of Drones. Institutional Report. Available online: https://www.iwm.org.uk/history/contemporary-conflict/a-brief-history-of-drones (accessed on 20 May 2026).
2. Wagner, W. *Lightning Bugs, and Other Reconnaissance Drones*; Aero Publishers, Inc.: Fallbrook, CA, USA, 1982.
3. International Civil Aviation Organization. *Annex II—Rules of the Air*, 11th ed.; ICAO: Montreal, QC, Canada, 2022.
4. International Civil Aviation Organization. *Unmanned Aircraft Systems Traffic Management (UTM)—A Common Framework with Core Principles for Global Harmonization*, 4th ed.; Guidance Document; International Civil Aviation Organization (ICAO): Montreal, QC, Canada, 2023.
5. European Union. Commission Implementing Regulation (EU) 2021/664 of 22 April 2021 on a Regulatory Framework for the U-space. Regulation (EU) 2021/664. Available online: https://eur-lex.europa.eu/eli/reg_impl/2021/664/oj (accessed on 20 May 2026).
6. EASA. Easy Access Rules for U-space (Regulation (EU) 2021/664). Available online: https://www.easa.europa.eu/en/document-library/easy-access-rules/easy-access-rules-u-space-regulation-eu-2021664 (accessed on 20 May 2026).
7. *ASTM F3548-21* ; Standard Specification for UAS Traffic Management (UTM) UAS Service Supplier (USS) Interoperability. ASTM: West Conshohocken, PA, USA, 2022.
8. Amarillo, S.; Sanchis, A.; Freire, B.F.; Balbastre, J.V. Proposal of UAS Strategic Conflict Detection Concept with a Centralised Service in Multi-USSP Environment Using an Octree Data Structure. In Proceedings of the 2025 Integrated Communications, Navigation and Surveillance Conference (ICNS), Brussels, Belgium, 8–10 April 2025; pp. 1–8. [CrossRef]
9. Obaid, L.; Hamad, K.; Al-Ruzouq, R.; Dabous, S.A.; Ismail, K.; Alotaibi, E. State-of-the-art review of unmanned aerial vehicles (UAVs) and artificial intelligence (AI) for traffic and safety analyses: Recent progress, applications, challenges, and opportunities. *Transp. Res. Interdiscip. Perspect.* **2025**, *33*, 101591. [CrossRef]
10. Sanchis, A.; Garcia, A.; Esparza, S.; Balbastre, J.V. Realistic Trajectory Generation in Simulated Environments for U-Space Systems Assessment. In Proceedings of the 2025 Integrated Communications, Navigation and Surveillance Conference (ICNS), Brussels, Belgium, 8–10 April 2025; pp. 1–9. [CrossRef]

11. Meng, W.; Zhang, X.; Zhou, L.; Guo, H.; Hu, X. Advances in UAV Path Planning: A Comprehensive Review of Methods, Challenges, and Future Directions. *Drones* **2025**, *9*, 376. [CrossRef]
12. Hart, P.E.; Nilsson, N.J.; Raphael, B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Trans. Syst. Sci. Cybern.* **1968**, *4*, 100–107. [CrossRef]
13. Choi, J.; Choi, Y. Path Planning for Unmanned Aerial Vehicle: A-Star-Guided Potential Field Method. *Drones* **2025**, *9*, 545. [CrossRef]
14. EUROCONTROL. Concept of Operations for European UTM Systems—Extension for Urban Air Mobility (CORUS-XUAM). Available online: https://www.eurocontrol.int/project/corus-xuam (accessed on 20 May 2026).
15. Cheriet, H.; Badra, K.K.; Samira, C. Comparative Analysis of UAV Path Planning Algorithms for Efficient Navigation in Urban 3D Environments. *arXiv* **2025**, arXiv:2508.16515. [CrossRef]
16. Harabor, D.; Grastien, A. Online Graph Pruning for Pathfinding On Grid Maps. *Proc. AAAI Conf. Artif. Intell.* **2011**, *25*, 1114–1119. [CrossRef]
17. Wu, Q.; Lin, H.; Jin, Y.; Chen, Z.; Li, S.; Chen, D. A new fallback beetle antennae search algorithm for path planning of mobile robots with collision-free capability. *Soft Comput.* **2020**, *24*, 2369–2380. [CrossRef]
18. Chen, D.; Li, X.; Li, S. A Novel Convolutional Neural Network Model Based on Beetle Antennae Search Optimization Algorithm for Computerized Tomography Diagnosis. *IEEE Trans. Neural Netw. Learn. Syst.* **2023**, *34*, 1418–1429. [CrossRef]
19. Li, Q.; Wan, L.; Xiao, B.; Zhou, Y. Research on Multi-Airspace Conflict Detection and Resolution Method Based on Octree Spatial Grid Partitioning. *Aerospace* **2025**, *13* , 15. [CrossRef]
20. Gordo, V.; Perez-Castan, J.A.; Sanz, L.P.; Serrano-Mira, L.; Xu, Y. Feasibility of Conflict Prediction of Drone Trajectories by Means of Machine Learning Techniques. *Aerospace* **2024**, *11*, 1044. [CrossRef]
21. He, Y.; Hou, T.; Wang, M. A new method for unmanned aerial vehicle path planning in complex environments. *Sci. Rep.* **2024**, *14*, 9257. [CrossRef]
22. Innan, N.; Kashif, M.; Marchisio, A.; Gan, Y.S.; Barbaresco, F.; Shafique, M. QUAV: Quantum-Assisted Path Planning and Optimization for UAV Navigation with Obstacle Avoidance. *arXiv* **2025**, arXiv:2508.21361. [CrossRef]
23. Mnih, V.; Kavukcuoglu, K.; Silver, D.; Graves, A.; Antonoglou, I.; Wierstra, D.; Riedmiller, M. Playing Atari with Deep Reinforcement Learning. *arXiv* **2013**, arXiv:1312.5602. [CrossRef]
24. Wang, Z.; Ng, S.X.; EI-Hajjar, M. Deep Reinforcement Learning Assisted UAV Path Planning Relying on Cumulative Reward Mode and Region Segmentation. *IEEE Open J. Veh. Technol.* **2024**, *5*, 737–751. [CrossRef]
25. Westheider, J.; Rückin, J.; Popović, M. Multi-UAV Adaptive Path Planning Using Deep Reinforcement Learning. *arXiv* **2023**, arXiv:2303.01150. [CrossRef]
26. Sharma, G.; Jain, S. Deep Reinforcement Learning-Based Framework for Path Planning of AUAVs. *Procedia Comput. Sci.* **2025**, *258*, 1112–1122. [CrossRef]
27. Jarray, R.; Bouallègue, S. Improved deep Q-network-based UAV path planning in partially observable environments. *Clust. Comput.* **2025**, *28*, 1046. [CrossRef]
28. Dierkes, J.; Cramer, E.; Hoos, H.H.; Trimpe, S. Combining Automated Optimisation of Hyperparameters and Reward Shape. *arXiv* **2024**, arXiv:2406.18293. [CrossRef]
29. EUROCONTROL. *Base of Aircraft Data (BADA) Aircraft Performance Modelling Report*; EEC Technical Report; EUROCONTROL Experimental Centre: Brétigny-sur-Orge, France, 2009.
30. Mendez, A.P.; Whidborne, J.F.; Chen, L. Wind preview-based model predictive control of multi-rotor uavs using LiDAR. *Sensors* **2023**, *23*, 3711. [CrossRef]
31. European Commission. *Commission Implementing Regulation (EU) 2019/947 of 24 May 2019 on the Rules and Procedures for the Operation of Unmanned Aircraft Systems*; Official Journal of the European Union; European Commission: Brussels, Belgium, 2019.
32. Gong, H.; Huang, B.; Jia, B.; Dai, H. Modelling Power Consumptions for Multi-rotor UAVs. *arXiv* **2022**, arXiv:2209.04128. [CrossRef]
33. Jacewicz, M.; Żugaj, M.; Głębocki, R.; Bibik, P. Quadrotor model for energy consumption analysis. *Energies* **2022**, *15*, 7136. [CrossRef]
34. ENAC. Lista Operatori Autorizzati ai sensi dei Regolamenti 2019/947 e 2018/1139 Art. 2, Comma 3(a). Available online: https://moduliweb.enac.gov.it/applicazioni/SAPR/APR_Lista_Unificata.asp (accessed on 20 May 2026).
35. Russell, S.J.; Norvig, P. *Artificial Intelligence: A Modern Approach*, 4th ed.; global edition ed.; Prentice Hall Series in Artificial Intelligence; Pearson: Boston, MA, USA, 2022.
36. Nobes, T.K.; Harabor, D.; Wybrow, M.; Walsh, S.D.C. The JPS Pathfinding System in 3D. *Proc. Int. Symp. Comb. Search* **2022**, *15*, 145–152. [CrossRef]
37. Sutton, R.S.; Barto, A.G. *Reinforcement Learning: An Introduction*; MIT Press: Cambridge, MA, USA, 1998.
38. Liang, S.; Srikant, R. Why Deep Neural Networks for Function Approximation? *arXiv* **2017**, arXiv:1610.04161. [CrossRef]

39. Vincent, F.L.; Peter, H.; Riashat, I.; Marc, G.B.; Joelle, P. An Introduction to Deep Reinforcement Learning. *Found. Trends® Mach. Learn.* **2018**, *11*, 219–354. [CrossRef]
40. Aliferis, C.; Simon, G. Overfitting, Underfitting and General Model Overconfidence and Under-Performance Pitfalls and Best Practices in Machine Learning and AI. In *Artificial Intelligence and Machine Learning in Health Care and Medical Sciences: Best Practices and Pitfalls*; Simon, G.J., Aliferis, C., Eds.; Springer International Publishing: Cham, Switzerland, 2024; pp. 477–524. [CrossRef]
41. Statnikov, A.; Aliferis, C.; Hardin, D.; Guyon, I. *A Gentle Introduction to Support Vector Machines in Biomedicine: Volume 1: Theory and Methods*; World Scientific: Singapore, 2011; p. 183. [CrossRef]
42. Silver, D.; Schrittwieser, J.; Simonyan, K.; Antonoglou, I.; Huang, A.; Guez, A.; Hubert, T.; Baker, L.; Lai, M.; Bolton, A.; et al. Mastering the game of Go without human knowledge. *Nature* **2017**, *550*, 354–359. [CrossRef] [PubMed]
43. Goodfellow, I.; Bengio, Y.; Courville, A. *Deep Learning*; MIT Press: Cambridge, MA, USA, 2016.
44. Kochenderfer, M.J.; Wheeler, T.A.; Wray, K.H. *Algorithms for Decision Making*; MIT Press: Cambridge, MA, USA, 2022.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.